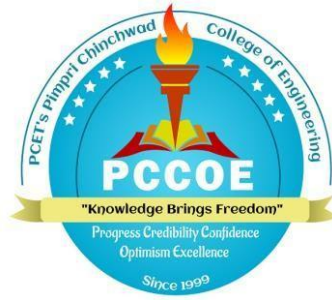




A

Project Report

On

Project ID: A03

“WildEcoWatch : An Integrated IoT and AI Framework for Wildlife Surveillance and Conservation”

Submitted

By

Sushant Arun Awate 121B1E007

Anuj Krishant Gavhane 121B1E056

Abhishek Dattatray Balgude 121B1E062

Under the Guidance of

Prof. V.A.Kulkarni

Department Of

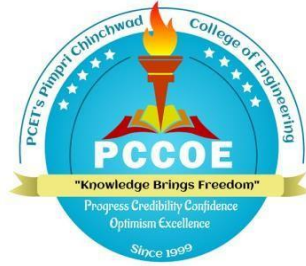
ELECTRONICS & TELECOMMUNICATION

**PIMPRI CHINCHWAD COLLEGE OF ENGINEERING
(SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE)**

2024-25

Project Scheme:A

CERTIFICATE



A
PROJECT REPORT
ON
Project ID:A03

“WildEcoWatch : An Integrated IoT and AI Framework for Wildlife Surveillance and Conservation”

Submitted for Partial Fulfillment of the Requirements for the Degree of Bachelor of Engineering in the Department of Electronics & Telecommunication Engineering
Pimpri Chinchwad College of Engineering, Savitribai Phule Pune University, Pune

By

- | | |
|---------------------------------------|------------------|
| 1) Sushant Arun Awate | 121B1E007 |
| 2) Anuj Krishnat Gavhane | 121B1E056 |
| 3) Abhishek Dattatraya Balgude | 121B1E062 |

Prof.V.A.Kulkarni

Prof.S.S.Ayane

Dr.K.S.Kinage

Dr.G.N.Kulkarni

ProjectGuide

ProjectCo-ordinator Head of Department
Pimpri Chinchwad College of Engineering,
Savitribai Phule Pune University, Pune

Director

2024-2025

Project Scheme A

CONTENT

Acknowledgment.....	1
Abstract.....	2
Table of Contents.....	3
List of Figures.....	4
List of Tables.....	5

ACKNOWLEDGMENT

Every achievement in the world is the result of collective effort, and this project is no exception. We extend our heartfelt gratitude to everyone who contributed to making this project a reality. We sincerely thank **Prof.V.A.Kulkarni** for their invaluable guidance and consistent supervision throughout the project. Their role as the project guide as well as the coordinator, along with her team's timely support and the necessary insights they provided, has been instrumental in our progress.

We are deeply appreciative of our Head of Department, **Dr. K. S. Kinage**, who's thoughtful reviews, constructive comments, and valuable suggestions greatly enriched our work. Our heartfelt thanks also go to our director, **Dr.G.N.Kulkarni**, for his unwavering encouragement and support.

In addition, we express our gratitude to our parents and friends for their constant inspiration and motivation, which have been a source of strength throughout this journey.

Lastly, we extend our sincere appreciation to the lab assistants and the Department of Electronics & Telecommunication for their active involvement at every stage of this project, ensuring its successful completion.

Sushant Awate

Anuj Gavhane

Abhishek Balgude

ABSTRACT

WildEcoWatch is an integrated IoT and AI framework designed for wildlife surveillance and conservation. The system utilizes motion detection sensors, camera modules, and AI-based object detection to monitor wildlife activity in real-time. When motion is detected, the system captures images, processes them using a pre-trained machine learning model to identify animal species, and logs the data for analysis. This automated approach helps in tracking wildlife movements without human intervention, making it valuable for conservation efforts in protected areas.

The core of WildEcoWatch consists of a Raspberry Pi-based IoT setup with a PIR (Passive Infrared) sensor to detect motion, triggering a camera to capture images. The captured images are analyzed using AI model, which predicts detected animals with confidence scores. If a recognized animal is found, the system activates visual and auditory alerts while logging the detection details in a CSV file for further study.

To enhance real-time monitoring, the system integrates a Flask-based web server that provides a user interface for manual image capture and detection. Additionally, email alerts are sent to conservationists or researchers when significant detections occur, ensuring prompt response to wildlife activity. The framework is designed to operate autonomously in remote locations, making it a cost-effective solution for wildlife monitoring.

By combining IoT sensors, AI-based detection, and automated logging, WildEcoWatch offers a scalable and efficient approach to wildlife surveillance. It reduces human dependency in data collection, minimizes disturbance to animals, and provides valuable insights for ecological research and habitat protection. This system can be deployed in national parks, sanctuaries, or research sites to aid in biodiversity conservation efforts.

Keyword: AI Framework, Biodiversity Protection, Camera Module, CSV Logging, Email Notifications, Flask Web Server, IoT Framework, Motion Detection, Object Detection, PIR Sensor, Raspberry Pi, Real-time Monitoring, Wildlife Conservation, Wildlife Surveillance, YOLOv8.

TABLE OF CONTENTS

Chapter 1: Introduction.....	
1.1 Motivation.....	
1.2 Background.....	
 Chapter 2: Literature Survey.....	
2.1. Literature Survey Table.....	
2.2. Summary of Literature Survey.....	
2.3 Gap Identified through Literature Survey.....	
2.4 Problem Statement.....	
2.4.1. Aim.....	
2.4.2 Objectives.....	
 Chapter 3: Methodology.....	
3.1 Project Outline.....	
3.2 Block Diagram Explanation.....	
3.3 Flow chart Explanation.....	
3.4 Algorithm Explanation.....	

Chapter 4: Design & Implementation of Proposed System.....

4.1 Circuit Diagram.....	
4.2 Hardware Specifications.....	
4.3 Design Consideration.....	
4.4 Hardware Implementation.....	

Chapter 5: Software Implementation.....

5.1 Software Required.....	
5.2 Software Specifications.....	
5.3 Implementation.....	

Chapter 6: Testing & Troubleshooting.....

6.1 Testing strategies	
6.2 Test Procedures.....	
6.3 Troubleshooting Procedures.....	
6.4 Results & Analysis.....	

Chapter 7: Advantages & Applications.....

7.1 Advantages.....	
7.2 Applications.....	

Chapter 8: Conclusion & Future scope.....

8.1 Conclusion.....

8.2 Future scope.....

Reference.....

Appendix:

Appendix A: Data sheets.....

Appendix B: Program Code.....

Appendix C : Bill of Materials (Component List & Costing).....

Appendix D : Research Publication (Publication/Patent/Copyright Proof).....

Appendix E : Project Participation Certificates.....

LIST OF FIGURES

Figure No.	Figure Name	Page No.
3.1	Block Diagram	22
3.2	Flowchart	23
3.3	Yolov8 Architecture	25
4.1	Circuit Diagram of Hardware	29
4.2	Hardware Schematic Diagram	33
4.3, 4.4	Hardware Implementation	34
5.3	Animal Detection - Leopard	39
6.1	Confusion Matrix	45
6.2, 6.3	Animal Detection - Lion and Elephant	46
6.4	Alerts - Email Notifications	47

LIST OF TABLES

Table No.	Table Name	Page No.
2.1	Literature Survey	16
4.1	Hardware Requirements	29
5.1	Software Requirements	36
6.1	Yolov8 Validation Results Summary	45

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
CSV	Comma-Separated Values
GPIO	General Purpose Input/Output
HTML	Hyper Text Markup Language
HTTP	Hyper Text Transfer Protocol
IoT	Internet of Things
LED	Light Emitting Diode
mAP	mean Average Precision
OS	Operating System
PIR	Passive Infrared
RAM	Random Access Memory
RPi	Raspberry Pi
UI	User Interface
USB	Universal Serial Bus
Wi-Fi	Wireless Fidelity
Yolov8	You Only Look Once Version 8

CHAPTER 1

CHAPTER 1

Introduction

1.1 Motivation

The WildEcoWatch system is designed to address the growing problem of intrusion, which harms wildlife and disrupts ecosystems in India. Current systems are not enough to detect intrusions in real-time, leaving wildlife at risk. This project aims to provide an affordable and effective solution using IoT devices and machine learning to detect such intrusions and send instant alerts. The system will help forest rangers, conservationists, and law enforcement to act quickly, improving their ability to protect animals and preserve forests. By preventing intrusion, this project will help protect endangered species and the environment for future generations.

1.2 Background

Wildlife conservation is crucial for maintaining biodiversity and ecological balance, but human activities like poaching and illegal logging pose significant threats to forests and endangered species. Traditional methods like manual patrols and basic camera traps are often inefficient and slow, leaving ecosystems vulnerable.

The proposed **WildEcoWatch** project uses modern technologies like IoT, machine learning, and embedded systems to create a smart surveillance solution for forests. By deploying devices such as cameras, motion sensors, and Raspberry Pi, it enables real-time monitoring and instant alerts to authorities when intrusions are detected.

This project falls under the category of **environmental conservation and embedded systems**. It aims to bridge the gap between technology and forest protection by providing an affordable, scalable, and efficient system that helps forest rangers and conservationists safeguard wildlife and ecosystems.

CHAPTER 2

CHAPTER 2

Literature Survey

2.1 Literature Survey Table

Sr. No	Title of the Paper	Year of Publication	Publisher	Methodology	Conclusion
1	Animal Intrusion Detection in Farming Area using YOLOv5 Approach	IEEE, ICCAS 2022	(1).Normaisharah Mamat, (2).Mohd Fauzi Othman, (3).Fitri Yakub.	Used YOLOv5 for detecting animal intrusions (elephant, monkey, tiger, wild boar) with a dataset of 280 images (70/class), resized to 416×416 and split into 80% training, 10% validation, and 10% testing. Evaluated using Precision, Recall, and mAP.	YOLOv5 achieved 94% mAP, 89% precision, and 89% recall in detecting animal intrusions, outperforming YOLOv4 in accuracy and efficiency
2	Animal Intrusion Detection Using Yolo V8	IEEE, ICACCS 2024	(1).Kathir M, (2).Balaji V , (3).Ashwini K	Implemented YOLOv8 for detecting four animals with 376 images per class. Achieved 99.4% mAP (training) and 95.9% (validation), outperforming earlier versions with mosaic augmentation and efficient convolutions.	YOLOv8 showed high accuracy and efficiency for real-time animal intrusion detection, aiding human-wildlife coexistence. Future work includes behavior analysis, real-time tracking, and community involvement
3	Implementation of a Wild Animal Intrusion Detection Model Based on Internet of Things	IEEE 2022	(1).Prabhat Kumar Panda, (2).Cherlopalli Srujan Kumar, (3).Bommu Sai Vivek, (4).Masura Balachandra, (5).Shashi Kant Dargar	The system uses an ESP32 camera, two Arduino Unos, and an E-vehicle. The transmitter detects intrusions via ultrasonic sensors and sends data using NRF24L01. The receiver controls the vehicle, triggering movement and image capture when an intrusion is detected	This paper presents an IoT-based wild animal intrusion detection system using Arduino Uno and sensors. Future improvements include integrating IR and wide-angle cameras for enhanced versatility.

4	Enhancing Farm Security: Animal Intrusion Detection using YOLO	IEEE Xplore 2023	(1). M. Venkata Krishna Rao. (2). Aakanksha Challawar (3). Begari Adithi. (4). Akhila Kurmandla. (5). Pranathi Bussa	The system uses YOLOv3 and OpenCV for animal detection, capturing images via high-sensitivity sensors. It processes images for feature extraction, classifies animals as harmful or harmless, and triggers alarms for harmful ones.	The system offers a safe, effective solution for monitoring agriculture, improving crop management and productivity. It tackles false alarms and accuracy issues, highlighting the need for regular updates and maintenance
5	Image Processing based Protection of Crops from Wild Animals using Intelligent Surveillance	IEEE ICEARS 2022	(1)A.Sathesh, (2)K.Vishnu, (3)A.Yuvaneshr, (4)V.Vellaisamy (5)K.Gowthami	The system uses cameras around farmlands with the YOLO algorithm for object detection. It processes images, extracts features, and compares them with predefined images. An Arduino and GSM module send alerts to farmers via alarms and messages, detecting and alerting about animal intrusions without harm.	The system captures and classifies intruding animals using YOLO, offering a low-cost, energy-efficient solution to protect crops. It helps prevent crop destruction and reduces financial losses for farmers.
6	Image Processing based Animal Intrusion Detection System in Agricultural Field using Deep Learning	IEEE 2024	(1). Oviya Gnanasekar. (2). Prapti Dinesh. (3). Srividhya K	(1).Utilizes Raspberry Pi and deep learning techniques, specifically YOLOv7, for real-time animal detection. (2)Captures live video feeds from agricultural fields and analyzes them to detect animal intrusions. (3).Emits specific repellent sounds and sends SMS alerts to farmers upon detection of intrusions.	YOLOv7 model demonstrates high accuracy and speed in detecting animal intrusions, contributing to the sustainability and productivity of agricultural practices. Future enhancements may include the use of drones and more advanced cameras for improved monitoring.

7	Enhancing Animal Intrusion Detection with Raspberry Pi and Fast R-CNN: A Practical Approach for Wildlife Management	IEEE 2024	(1).N Kanthimathi (2). R Sheela Daniel (3). R. Santhana Krishnan. (4). J. Samuel Prabakaran (5). S. Gopikumar (6). V. Sakthi Maragatham	The system integrates Raspberry Pi with a camera, thermal, and motion sensors, using Fast R-CNN for local video processing. It compares Fast R-CNN with YOLO and SSD models across three datasets, evaluating accuracy, precision, recall, and F1 score.	The proposed animal intrusion detection system ensures real-time, reliable protection of agricultural and wildlife areas, with Fast R-CNN outperforming YOLO and SSD models in detection accuracy and response.
8	Animals Detection System In The Farm Area Using Iot	ICCCI 2023	(1).Ms.K.Chitra (2).Dr.A.Tamilarasi (3).Dr.M.Pyingkodi (4)K. Nanthini	The system uses Arduino Uno with ultrasonic sensors to detect animal intrusions and sends alerts via a GSM module through calls and alarms. It monitors continuously, triggering alarms at specified heights until the farmer responds.	The system protects crops from wild animals, reducing losses and allowing farmers to monitor fields without constant supervision. It's affordable, operates 24/7, and enhances crop protection through IoT technology.
9	Animal Detection and Classification in Image & Video Frames using YOLOv5 and YOLOv8	IEEE 2023	(1). Aishwarya D Shetty (2) Soumya Ashwath	(1)Developed a machine learning system for automatic animal detection in surveillance footage using YOLOv5 and YOLOv8. (2)Utilized the COCO dataset for training and testing the models. (3)Implemented a user interface for inputting images or videos, including live input options. (4)Conducted pre-processing of images and created a dataset.	(1).YOLOv8 achieved higher accuracy (around 85%) compared to YOLOv5, making it more suitable for applications requiring high precision. (2)YOLOv5, while faster and smaller, had lower accuracy, making it more appropriate for real-time applications where speed is crucial. The choice between YOLOv5 and YOLOv8 should (depend on specific application requirement

10	Advanced Wild Animal Detection and Alert System using YOLO V5 Model	IEEE 2023	(1)C.S.Madhumathi (2) Mr.V.Naveen (3) Mr.Nallamala Akshay (4) Mr.M.Sanjay Kumar	(1)Utilizes the YOLO V5 object detection algorithm for real-time wild animal detection. (2)Employs cameras to capture video feeds, which are processed to identify wild animals. (4)Incorporates a user-friendly interface for monitoring and alerts. (5)Evaluates system performance using metrics like recall, accuracy.	(1)The proposed system offers a cost-effective solution for detecting and tracking wild animals, enhancing community safety and environmental well-being. (2).Future work aims to improve accuracy, integrate with other systems, and expand deployment to diverse regions.
11	Wild Animal Detection using Deep Learning	IEEE 2022	(1). Sreedevi K L (2). Dr Anitha Edison	The model uses depth-wise separable convolution layers and zero padding to maintain edge characteristics. It was tested on the IWildCam dataset and evaluated using IOU, precision, recall, F1 score, and confusion matrix	The algorithm detects and classifies wild animals with 99.6% accuracy and an IOU of 0.878. The results show strong performance, with potential for improvement using multiple sensors.
12	Crops Protection System from Animals using Arduino	IEEE 2021	(1)P.Manikandn (2) A.Thenmozhi (3). G. Ramesh (4). T. Ravi Kiran Naidu. (5). K. Vamsinath Reddy. (6).K.BhanuPra deep Kumar Reddy	(1)Utilizes Arduino, PIR motion sensor, buzzer, LED lights, and GSM module. (2)Detects animal movement using PIR sensor. (3)Activates buzzer and LED lights upon detection. (4)Sends SMS alerts to the farmer within 10 seconds of detection. (5)Designed to protect crops from animal interference without harming animals or humans.	The system alerts farmers to animal intrusions, preventing crop damage and financial losses. It's practical, affordable, and improves crop yields. Future improvements may include cellular networks and additional sensors for better monitoring.

13	Real-Time Monitoring of Agricultural Land with Crop Prediction and Animal Intrusion Prevention using Internet of Things and Machine Learning at Edge	IEEE 2020	(1). Nikhil R (2). Prof. Anisha B (3). Dr. Ramakanth Kumar P	(1)Utilizes IoT devices for smart irrigation based on soil moisture, humidity, and temperature. (2)Implements machine learning techniques for crop prediction based on soil parameters. (3)Employs ultrasonic sound for animal intrusion prevention and alerts farmers via SMS and Email notifications. (4)Data collected from sensors is sent to the cloud for analysis and decision-making.	(1).The proposed system is cost-effective, enhances agricultural water management, aids in crop prediction, and prevents wild animal intrusions. (2).It automates irrigation processes based on real-time soil conditions, thereby improving efficiency and reducing resource wastage.
14	Intrusion Detection and Repellent System for Wild Animals Using Artificial Intelligence of Things	IEEE 2022	(1). Hardiki Deepak Patil (2). Dr. Namrata Farooq Ansari	(1).Utilizes Raspberry Pi for movement detection and video capture. (2)Implements YOLOv3 for real-time animal detection. (3)Employs PIR sensors to detect motion and trigger the camera. (4.)Activates a buzzer and flashlight to repel detected wild animals without harm.	(1).The proposed system offers an innovative solution to mitigate human-wildlife conflicts by providing a safe and automated method to detect and repel wild animals, thereby protecting both human habitats and wildlife.

15	Animal Intrusion Detection and Ranging system using YOLOv4 and LoRa	IEEE 2022	(1). Dr BMeenakshi (2). Aouthithiye Barathwaj SR Y (3). Haariharan N C (4). Krishnakanth L (5). Abishek J	(1)Integrated system using YOLOv4 for animal detection and LoRa for communication. (2)Utilizes Raspberry Pi for image processing and ESP-32 for communication. (3)Employs K-means clustering for predicting animal locations based on past data.	(1).The proposed system effectively addresses man-animal conflict by providing timely alerts to farmers and wildlife authorities. (2).It aims to create a sustainable ecosystem by minimizing property and life loss for both humans and animals. Successful localization of wild animals enhances response strategies to prevent conflicts.
----	---	-----------	---	--	--

Table 2.1 Literature Survey

2.2 Summary of Literature Survey

The literature survey highlights the critical need for advanced, integrated systems to address wildlife intrusion and poaching, particularly in ecologically sensitive areas. Traditional methods like manual patrols and basic camera traps are often inefficient, slow, and lack real-time response capabilities. Recent advancements in IoT, AI, and deep learning offer scalable solutions, but challenges such as hardware limitations, false alarms, and power constraints persist.

Several studies demonstrate the effectiveness of AI-driven detection models. For instance, YOLOv5 achieved a mean average precision (mAP) of 94% in detecting animals like elephants and tigers, while YOLOv8, an anchor-free model, outperformed earlier versions with a mAP of 99.4% on training data and 95.9% on validation data. Fast R-CNN showed higher accuracy in thermal sensor-based detection but required more computational resources. These models are particularly suited for edge deployment due to their efficiency and precision.

IOT and sensor networks have also been explored for wildlife monitoring. Systems using ESP32 cameras and ultrasonic sensors provided real-time intrusion alerts but lacked AI integration. Raspberry Pi paired with PIR sensors enabled 24/7 monitoring, though with limited threat classification. LoRa-based systems with YOLOv4 offered long-range alerts, addressing connectivity issues in remote areas. However, many existing systems rely on GSM modules for alerts, which can be unreliable in low-signal zones.

Sustainability is another key focus, with solar-powered sensors reducing reliance on grid electricity. Despite these advancements, gaps remain in real-time processing, multi-modal alerts, and scalability. High-cost hardware like thermal cameras limits widespread deployment, and many systems fail to integrate SMS, voice alerts, or augmented reality (AR) for field personnel.

The proposed WildEcoWatch system addresses these gaps by combining edge AI (YOLOv8n on Raspberry Pi) for low-latency detection, multi-channel alerts (SMS, email via MailJet), and sustainable solar-powered designs. This synthesis of technologies aims to deliver a scalable, cost-effective, and automated solution for wildlife conservation, bridging the divide between cutting-edge innovation and practical ecological needs. The system's modular design and energy-efficient components further enhance its suitability for remote deployments, making it a promising tool for combating wildlife crimes and preserving biodiversity.

2.3 Gap Identified through Literature Survey:

1. **Limited Real-Time Processing:** Many current systems rely on cloud-based AI processing, causing delays in threat detection and response. Edge computing solutions are needed for faster, on-site analysis.
2. **Lack of Multi-Modal Alerts:** Most systems use basic GSM or buzzer alerts, which fail in low-network areas. There is a need for integrated SMS, email (MailJet), and voice-based notifications for reliable communication.
3. **High False Alarm Rates:** Traditional PIR and ultrasonic sensors often trigger false alarms due to environmental factors (e.g., moving foliage). AI-powered object classification (YOLOv8n) can improve accuracy but is not widely implemented.
4. **Power Dependency:** Many systems rely on grid electricity, making them unsuitable for remote forests. Solar-powered, energy-efficient designs are necessary for sustainable operation.
5. **Scalability Issues:** High-cost hardware (e.g., thermal cameras) limits large-scale deployment. Low-cost alternatives like Raspberry Pi with optimized AI models (YOLOv8n) offer a more scalable solution.
6. **Insufficient Data Logging & Analytics:** Few systems provide structured data storage (cloud/local) for long-term wildlife behavior analysis, which is crucial for conservation strategies.
7. **Lack of Proactive Deterrents:** Most solutions focus only on detection rather than non-invasive deterrents (ultrasonic/LED repellents) to prevent intrusions without harming animals.
8. The WildEcoWatch project addresses these gaps by integrating edge AI (YOLOv8n), multi-channel alerts, solar power, and modular IoT sensors to create a real-time, scalable, and sustainable wildlife monitoring system.

2.4 Problem Statement

Wildlife crime, especially intrusion, poses a serious threat to ecosystems and communities in India, with inadequate detection systems in place to prevent such activities. This makes it difficult to protect wildlife and maintain environmental balance. The proposed WildEcoWatch system aims to solve this by using IoT devices, machine learning, and real-time alerts to detect intrusions in rainforests. By deploying an intelligent surveillance system at key locations, the system can notify authorities instantly, providing a more efficient and affordable solution to protect wildlife

2.4.1 Objectives

1. **Develop an Intelligent Surveillance System:** Create a real-time monitoring system using Raspberry Pi, PIR sensors, and HD cameras to detect poaching and ecological disruptions in rainforests.
2. **Implement Advanced AI Detection:** Utilize YOLOv8n machine learning model with OpenCV for high-accuracy animal and human intrusion detection.
3. **Establish Real-Time Alert Framework:** Deploy MailJet API for instant email/SMS notifications to authorities.
4. **Ensure Sustainable Operation:** Design solar-powered with energy-efficient components for off-grid functionality. Optimize power consumption for 24/7 monitoring capability
5. **Create Scalable Architecture:** Develop modular hardware design for easy deployment across diverse ecosystems.
6. Implement cloud/local (Excel) data storage for flexible resource management

2.4.2 Aim:

The aim of this project is to develop a solar-powered, intelligent surveillance system using Raspberry Pi, PIR sensors, and HD cameras, integrated with YOLOv8n and OpenCV for real-time animal and human detection. The system will ensure ecological protection in rainforest environments through automated alerts via the MailJet API and flexible data storage. It is designed to be energy-efficient, modular, and scalable for continuous, off-grid monitoring and deployment across various ecosystems

CHAPTER 3

CHAPTER 3

Methodology

3.1 Project Outline

The “**WildEcoWatch**” project presents an innovative approach to wildlife surveillance and conservation by combining **Internet of Things (IoT)** and **Artificial Intelligence (AI)** technologies. The system is primarily designed to operate autonomously in forest or reserve areas, detecting wildlife movement and capturing visual evidence for analysis and alerts. The entire workflow is driven by a **Raspberry Pi 4B**, which serves as the central control and processing unit.

The process begins with a **PIR (Passive Infrared) motion sensor** that continuously monitors the environment for any motion. Upon detecting movement, it sends a signal to the Raspberry Pi to initiate image capture. A **USB webcam** is activated to capture a clear image of the area. This image is then processed by a pre-trained **YOLOv8 AI model** deployed via the **Roboflow API**, which detects and classifies any animal present in the frame. The model filters detections based on a confidence threshold to ensure only relevant animals are logged and notified.

Once an animal is positively identified, the Raspberry Pi activates an **LED** and **buzzer** to provide real-time local alerts. Simultaneously, the system sends an **email notification** to a registered recipient using the **Mailjet API**, including details such as the animal type and confidence level. To facilitate data analysis and conservation efforts, each detection is also logged into a **CSV file**, storing the timestamp, species detected, and confidence value.

A user-friendly **web interface** is hosted on the Raspberry Pi using **Flask**, enabling manual image capture and viewing of the latest detections from a connected device. This web-based control panel adds interactivity and flexibility for researchers or forest officials.

The system is built for continuous, real-time monitoring and can operate independently in remote locations with minimal human intervention. By automating wildlife tracking and reporting, **WildEcoWatch** aims to improve conservation efficiency, reduce human-wildlife conflicts, and support ecological studies with real-time, AI-powered insights.

3.2 Explanation of Block Diagram

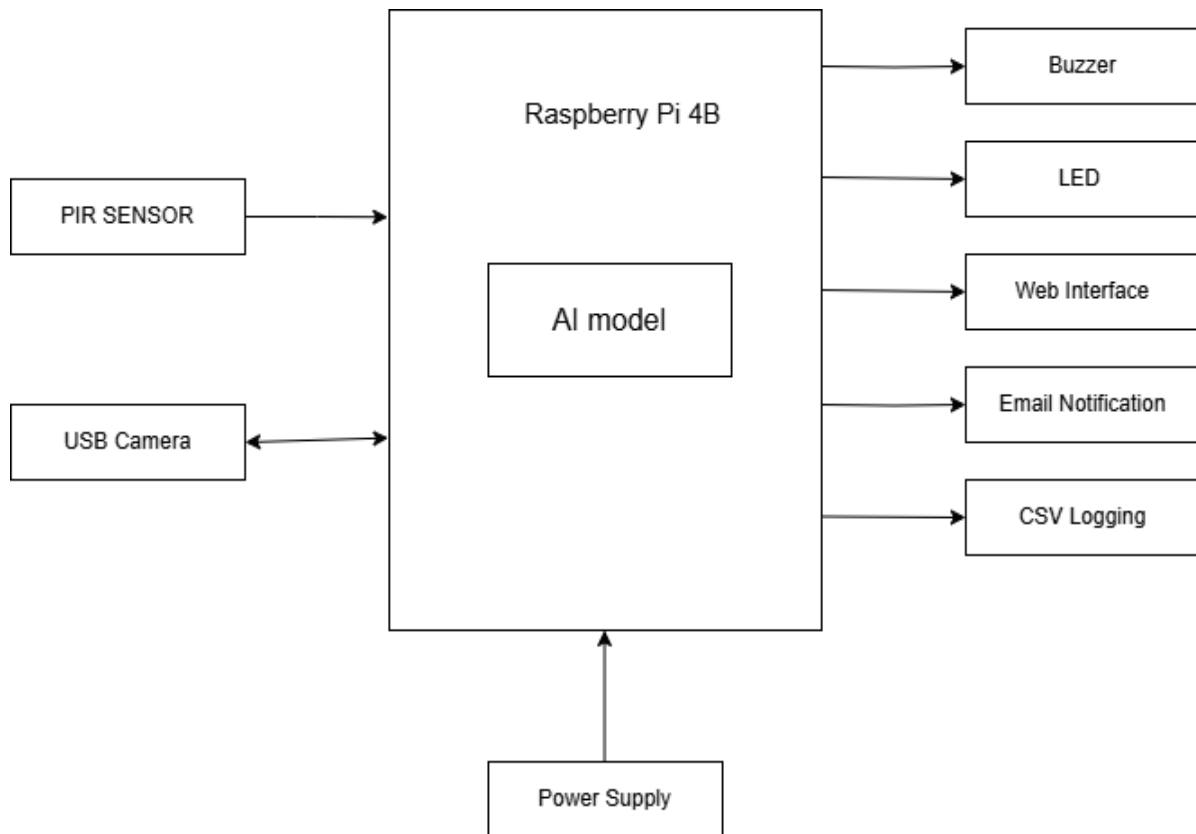


Fig.3.1 Block Diagram

The block diagram (Fig 3.1) illustrates the core architecture of the “WildEcoWatch” system, consisting of three major functional units: Sensing and Image Capture, Processing and Intelligence, and Output & Alert Mechanisms, all integrated with a Raspberry Pi 4B as the central controller.

1. Unit 1: Sensing and Image Capture

A PIR sensor constantly scans for motion in the environment. Once motion is detected, it activates a USB camera, which captures an image of the area. This image is then sent for analysis.

2. Unit 2: Processing and Intelligence

The Raspberry Pi 4B processes the captured image using an AI model (YOLOv8) accessed via the Roboflow API. Based on detection confidence, it decides whether an animal is present. It also handles logic, data logging, email alerts, and provides access to a web interface for real-time interaction.

3. Unit 3: Output & Alerts

If an animal is detected, a buzzer and LED are activated for local alerts. Simultaneously, an email notification is sent using the Mailjet API, and detection data is saved in a CSV file. The web interface allows users to manually control the system and review captured results.

All components are powered by a 5V/3A power supply, ensuring continuous, efficient operation in remote environments.

3.3 Explanation of Flowchart

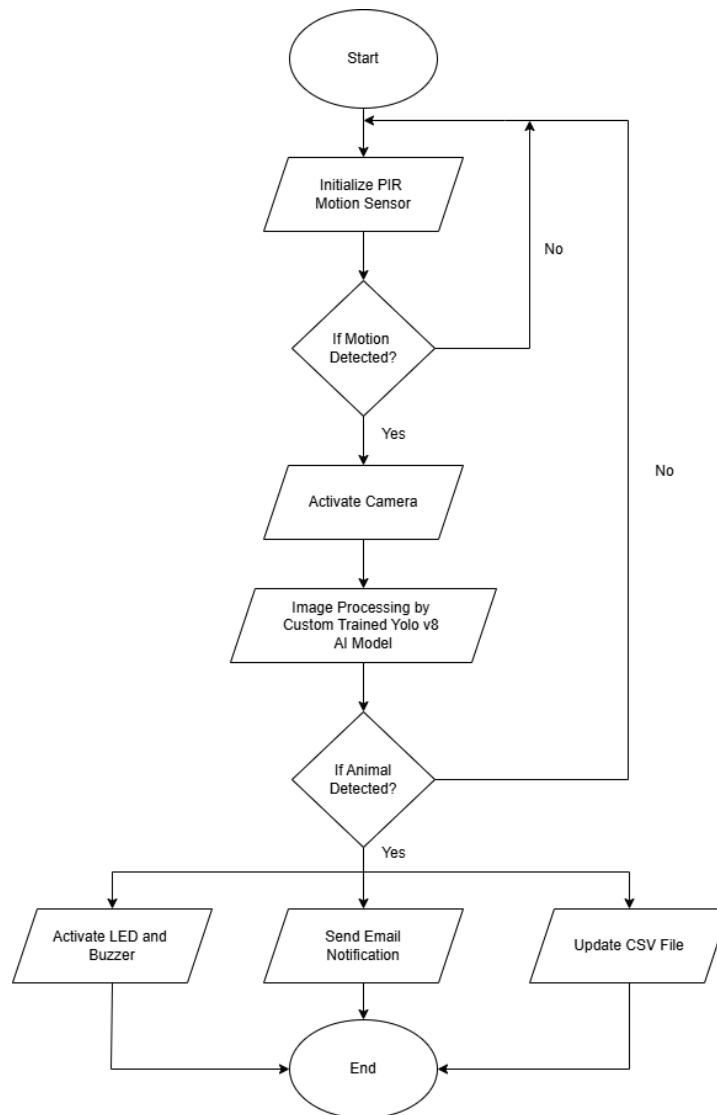


Fig.3.2 Flowchart

Steps:

1. Start

The system begins by initializing all hardware and software components, ensuring they are ready for operation.

2. Initialize PIR Motion Sensor

The Passive Infrared (PIR) sensor is activated to detect motion within its range. This sensor is the primary trigger for the system.

3. If Motion Detected?

The system checks if the PIR sensor has detected any motion.

- Yes: Proceeds to the next step (Activate Camera).
- No: The system remains idle, continuously monitoring for motion.

4. Activate Camera

Upon motion detection, the camera module is triggered to capture an image of the scene where motion was detected.

5. Image Processing by Custom Trained YOLOv8 AI Model

The captured image is processed using a pre-trained YOLOv8 model, which is optimized for wildlife detection. The model analyzes the image to identify and classify any animals present.

6. If Animal Detected?

The system checks if the AI model has identified an animal in the image.

- Yes: Proceeds to activate alerts and log data.
- No: The system returns to the monitoring state.

7. Activate LED and Buzzer

If an animal is detected, the system triggers visual (LED) and auditory (buzzer) alerts to notify nearby personnel or deter the animal if necessary.

8. Send Email Notification

An email alert is sent to predefined recipients (e.g., researchers or conservationists) containing details of the detected animal, such as species and confidence score.

9. Update CSV File

The detection details (timestamp, animal species, confidence score) are logged into a CSV file for future analysis and reporting.

10. End

The process concludes, and the system resets to continue monitoring for new motion

3.4 Algorithm Explanation:

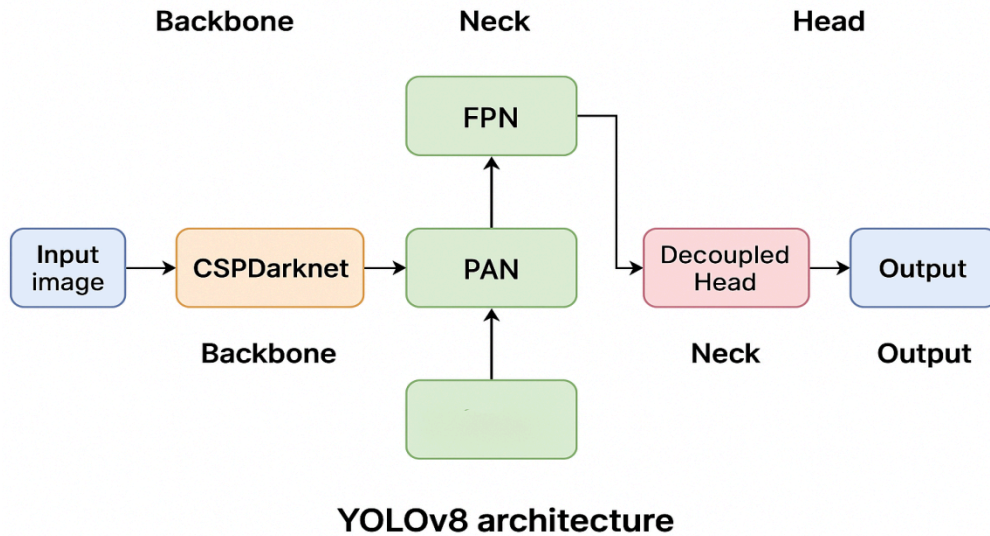


Fig.3.3 YOLOv8 Architecture

YOLOv8 Algorithm & Methodology

Introduction: YOLOv8 (You Only Look Once version 8) is the latest iteration of the YOLO family of real-time object detection algorithms. Developed by Ultralytics, YOLOv8 incorporates numerous improvements in performance, accuracy, and flexibility compared to its predecessors. It supports both object detection and instance segmentation tasks.

Methodology:

1. Architecture:

YOLOv8 uses a backbone-head architecture. The backbone extracts features from the input image, and the head performs object detection or segmentation. YOLOv8 employs a custom backbone optimized for speed and accuracy.

3.4.1.1 Backbone: Feature Extraction

Input Image

The architecture starts with an input image, which is typically resized to a standard dimension (e.g., 640x640) before processing.

CSPDarknet (Cross Stage Partial Darknet)

Role: Extracts essential visual features from the input image.

Components:

- **Convolutional layers:** Capture patterns like edges and textures.
- **CSPNet (Cross Stage Partial Network):** Enhances learning by splitting the input into two paths, processing one, and merging them later. This reduces computation and improves gradient flow.
- **Advantages:**
 - Reduces inference time.
 - Boosts learning capability by allowing better feature reuse.

3.4.1.2. Neck: Feature Aggregation

PAN (Path Aggregation Network)

- **Role:** Aggregates features from different backbone levels.
- **Function:**
 - Passes lower-layer features (high-resolution) upward and higher-layer features (low-resolution, rich in semantics) downward.
 - Helps the model detect objects at various scales (e.g., small animals like birds or large ones like elephants).

FPN (Feature Pyramid Network)

- **Role:** Combines features from different levels to enhance multi-scale detection.
- **Function:**
 - Constructs feature maps at multiple resolutions.
 - Supports better detection of small and large objects alike.

3.4.1.3. Head: Prediction

Decoupled Head

- **Role:** Separates classification and localization tasks.
- **Why "Decoupled"?:**
 - Traditional YOLO heads used shared layers for classification (what is it?) and localization (where is it?).
 - YOLOv8 separates them to improve specialization and accuracy in both tasks.

Output

- The final layer outputs:
 - **Bounding boxes:** Location of detected objects.
 - **Class probabilities:** What the object is.
 - **Objectness score:** Confidence that a box contains an object.

2. Training Process:

YOLOv8 is trained using annotated datasets where objects are labeled with bounding boxes and class labels. The training process involves optimizing the loss function that combines classification loss, localization loss (bounding box regression), and objectness loss.

3. Loss Function:

YOLOv8 uses a compound loss function that includes:

- **Box loss:** Measures how accurately the predicted boxes match the ground truth
- **Objectness loss:** Indicates whether an object is present in the predicted box
- **Classification loss:** Ensures correct class prediction for detected objects

4. Inference:

During inference, YOLOv8 takes an input image and passes it through the network to predict bounding boxes and class labels. Post-processing techniques such as Non-Maximum Suppression (NMS) are used to remove overlapping boxes and retain the most confident predictions.

Key Features of YOLOv8 Architecture:

- **Anchor-free detection:** Instead of predefined anchor boxes, it directly predicts object centers and sizes, making it faster and more flexible.
- **Decoupled Head:** Improves performance by separating object classification and localization.
- **Efficient:** Optimized for real-time performance even on edge devices like the Raspberry Pi or Jetson Nano.

Conclusion:

YOLOv8 represents a significant advancement in real-time object detection, balancing high accuracy with fast inference speeds. Its modular design and support for various tasks like classification, detection, and segmentation make it suitable for diverse computer vision applications.

CHAPTER 4

CHAPTER 4

Design & Implementation of Proposed System

4.1 Circuit Diagram

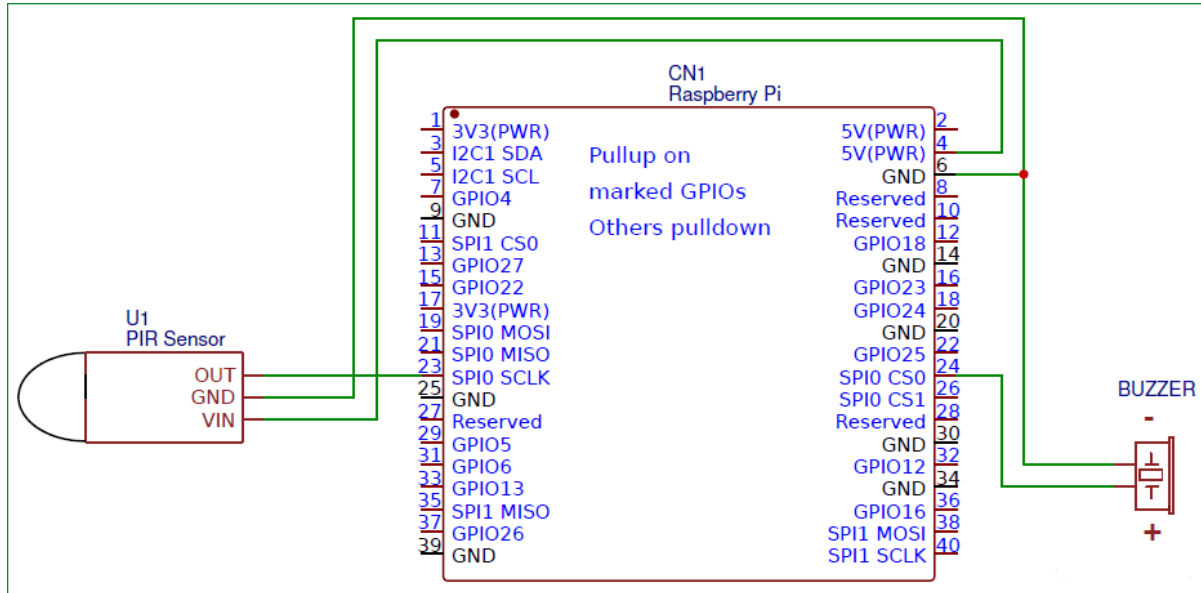


Fig.4.1 Circuit Diagram of Hardware

4.2 Hardware Specifications

Component	Specification / Role
Raspberry Pi 4	Central controller that runs the code and connects to sensors
PIR Motion Sensor	Detects motion in the monitored area
USB Webcam	Captures images upon motion detection
LED	Acts as a visual alert when an animal is detected
Buzzer	Provides audible alert upon detection
Jumper Wires	Used to connect components to GPIO pins
Breadboard	For prototyping and organizing sensor connections
Power Supply	To power the Raspberry Pi and connected components
MicroSD Card	Stores the OS and codebase

Table 4.1Hardware Specifications

1. Raspberry Pi 4 (Model B):

Function:

- Acts as the central processing unit to interface with sensors (PIR, webcam) and output devices (LED, buzzer).
- Runs Python scripts to detect motion, capture images/videos, and trigger alarms.

Specifications:

- **CPU:** Broadcom BCM2711, Quad-core Cortex-A72 (1.5GHz)
- **RAM:** 2GB, 4GB, or 8GB LPDDR4 options
- **Connectivity:** Dual-band Wi-Fi (2.4GHz/5GHz), Bluetooth 5.0, Gigabit Ethernet
- **Ports:** 2 × USB 3.0, 2 × USB 2.0, 2 × Micro HDMI, 3.5mm audio jack
- **GPIO Pins:** 40-pin header (for PIR, LED, buzzer interfacing)
- **Power:** 5V/3A via USB

2. PIR (Passive Infrared) Motion Sensor:

Function:

- Detects motion based on infrared heat signatures (human/animal movement).
- Sends a signal to the Raspberry Pi when motion is detected.

Specifications:

- **Operating Voltage:** 5V–20V (typically 5V)
- **Detection Range:** Adjustable (3m–7m)
- **Output Signal:** Digital (High/Low)
- **Delay Time:** Adjustable (0.3s–5min)

3. USB Webcam:

Function:

- Captures images or video when motion is detected for surveillance.
- Can integrate with OpenCV or fswebcam for recording

Specification:

- **Resolution:** 720p (HD) or 1080p (Full HD)
- **Interface:** USB 2.0/3.0
- **Focus:** Fixed or adjustable
- **Compatibility:** Works with Raspberry Pi OS (Linux UVC drivers)

4. LED (5mm, Red/Green):

Function:

- Visual indicator for motion detection (e.g., turns on when PIR detects movement).

Specifications:

- **Forward Voltage:** ~2V (Red), ~3.3V (Green)
- **Current:** 20mA (use a 220Ω resistor with 5V GPIO)

5. Buzzer (Active/Piezo)

Function:

- Emits an audible alarm when motion is detected.

Specifications:

- **Type:** Active (3.3V–5V) or Passive (requires PWM)
- **Operating Voltage:** 3V–12V (typically 5V)

6. Jumper Wires (Male-to-Female, Male-to-Male)

Function:

- Connects components (PIR, LED, buzzer) to the Raspberry Pi GPIO pins.

Specifications:

- **Type:** Dupont wires
- **Length:** 10cm–30cm

7. Breadboard :

Function:

- Provides a prototyping platform for temporary circuit connections.

Specifications:

- **Voltage Rating:** 5V–12V

8. Power Supply (5V/3A USB-C):

Function:

- Powers the Raspberry Pi 4 reliably (avoids undervoltage warnings).

Specifications:

- **Output:** 5V DC, 3A
- **Connector:** USB-C

9. MicroSD Card (16GB–64GB, Class 10):

Function:

- Stores the Raspberry Pi OS and recorded images/videos.

Specifications:

- **Capacity:** $\geq 16\text{GB}$ (32GB recommended)
- **Speed Class:** Class 10 (for smooth OS operation)

4.3 Design Consideration

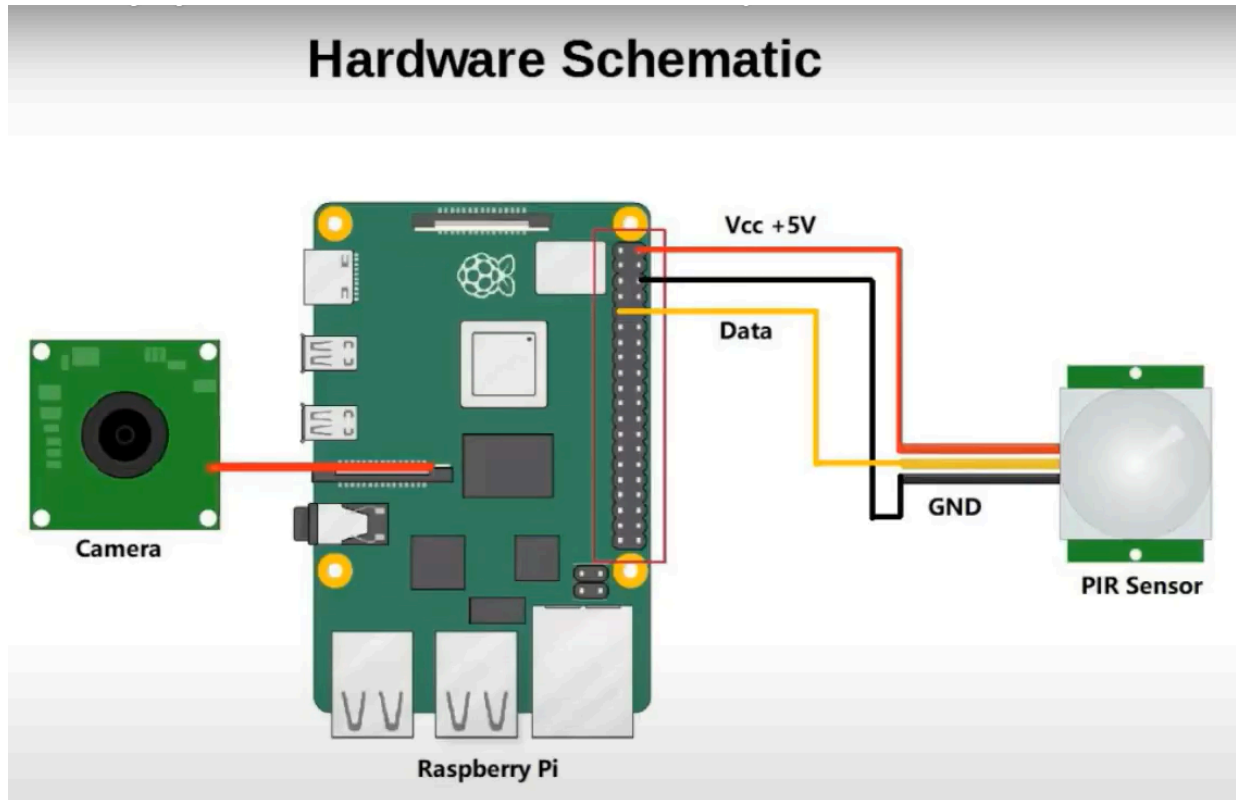


Fig 4.2 Hardware Schematic Diagram

The schematic (Fig 4.2) shows how the **PIR motion sensor** and **camera module** are connected to the **Raspberry Pi 4B** for real-time wildlife monitoring.

- **Camera Module:**
Connected to the Pi's dedicated camera port, it captures images when triggered by motion. These images are analyzed by the AI model for animal detection.
- **PIR Sensor:**
Detects motion and sends a signal via its **Data pin** (yellow) to the Pi.
 - **VCC (Red):** Connected to 5V power
 - **GND (Black):** Connected to ground
 - **Data (Yellow):** Connected to a GPIO input pin

Once motion is detected, the camera is activated, and the captured image is processed by the AI system. This setup ensures efficient and reliable field-based surveillance.

4.4 HARDWARE IMPLEMENTATION:

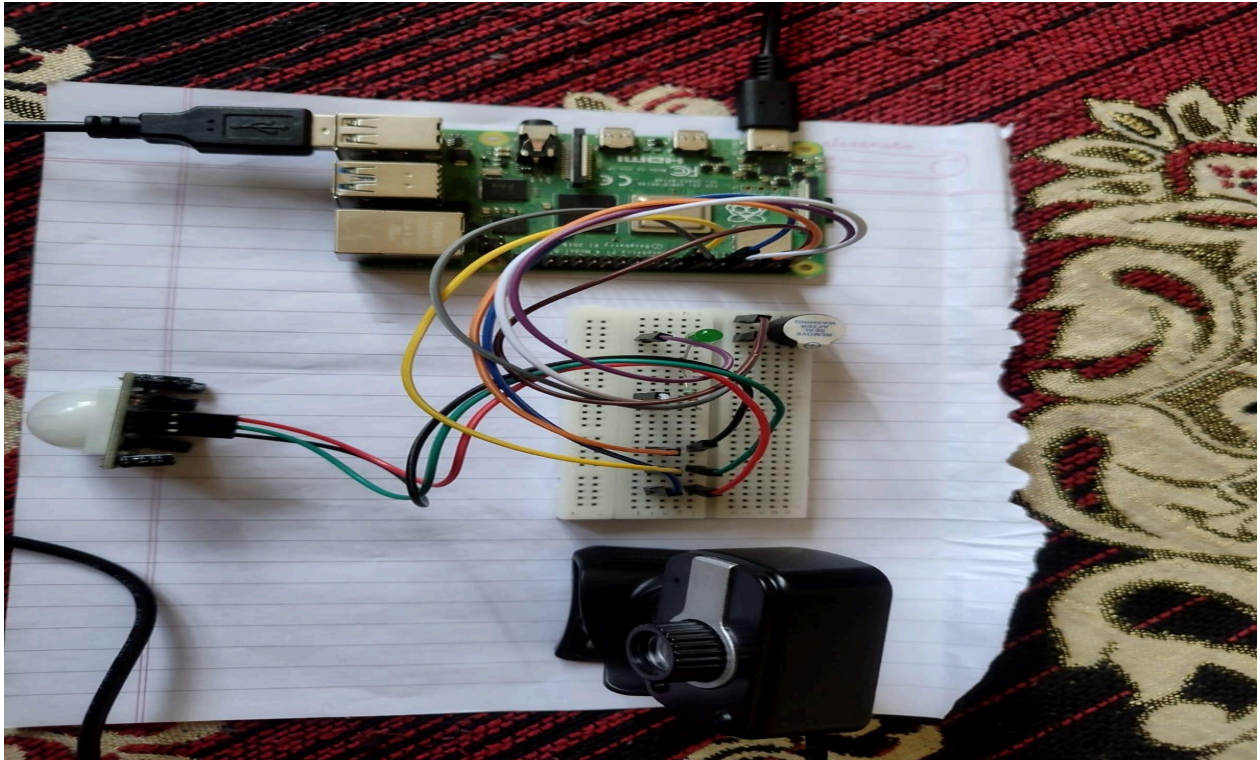


Fig.4.3 Hardware Implementation

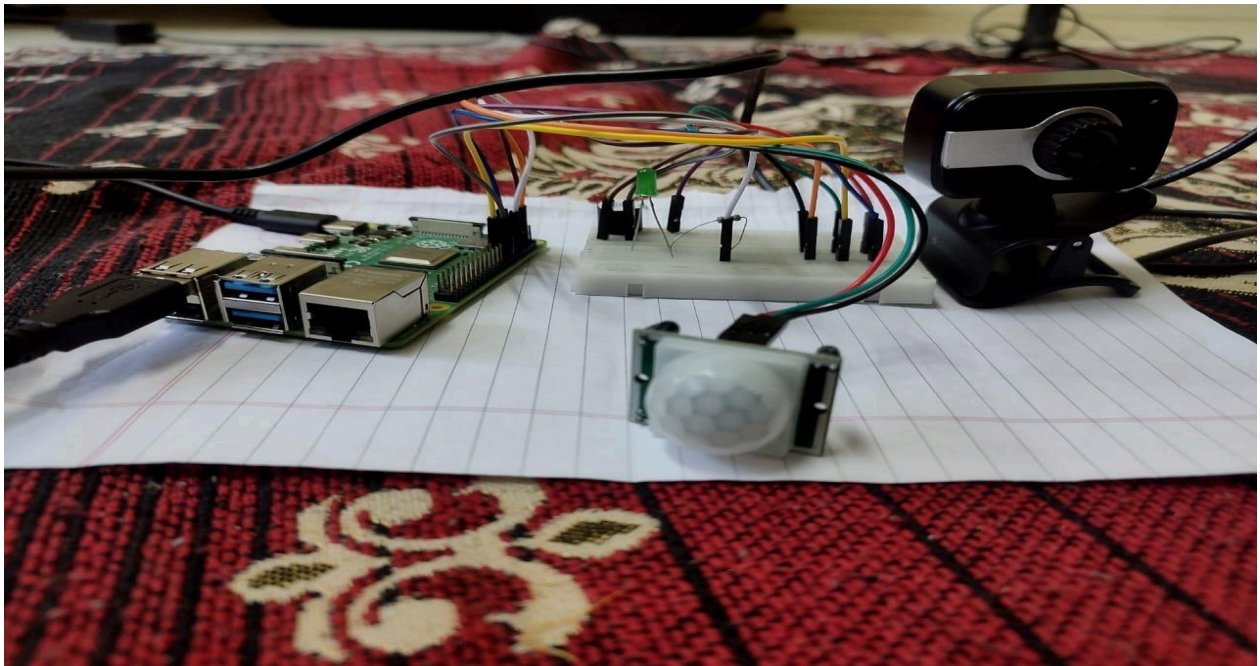


Fig.4.4 Hardware Implementation

CHAPTER 5

CHAPTER 5

Software Implementation

5.1 Software Requirements

Component	Purpose
Raspberry Pi OS	Operating system running on Raspberry Pi
Python 3	Core programming language used for scripting and logic
Flask	Backend web framework to handle API and serve frontend
OpenCV (cv2)	For capturing and processing images from the webcam
Mailjet REST API	To send email alerts upon animal detection
Pandas	For handling and logging data in CSV format
Threading	For running motion detection in the background
RPi.GPIO	To interface with hardware components via GPIO pins
HTML/CSS/JavaScript	For the frontend user interface served by Flask

Table 5.1 Software Requirements

5.2 Software Specifications

1. Raspberry Pi OS

Function:

- Serves as the primary operating system for running the entire application on the Raspberry Pi.

Specifications:

- Debian-based OS optimized for Raspberry Pi hardware.
- Supports Python, GPIO, camera interfacing, and networking.
- Allows SSH and remote access for headless operation.

2. Python 3

Function:

- Core programming language for writing logic, controlling devices, and building the backend.

Specifications:

- Used for scripting detection logic, image processing, hardware interfacing, and web backend.
- Supports multithreading for handling concurrent tasks (e.g., motion detection + image processing).

3. Flask

Function:

- Lightweight backend framework to build and run the web server and APIs.

Specifications:

- Serves frontend HTML/JS files.
- Exposes endpoints for image prediction and other interactions.
- Handles incoming requests and responses.

4. OpenCV (cv2)

Function:

- Captures video/images from the USB webcam and processes them for prediction.

Specifications:

- Reads frames from webcam using cv2.VideoCapture.
- Can resize, crop, or preprocess images before feeding to the YOLO model.
- Converts images to appropriate format (e.g., numpy array) for inference.

5. Mailjet REST API

Function:

- Sends email alerts when an animal is detected.

Specifications:

- Uses HTTP POST requests with API keys.
- Sends image attachments or detection details in the email body.
- Can include timestamp and location (if applicable).

6. Pandas

Function:

- Logs detection results into CSV or Excel format for tracking and analysis.

Specifications:

- Creates and appends to a DataFrame.

- Stores data like timestamp, animal type, image path, etc.
- Exports logs in .csv or .xlsx for easy access and review.

7. Threading

Function:

- Runs motion detection or sensor input monitoring in parallel with other tasks.

Specifications:

- Ensures the main Flask app remains responsive.
- Can run PIR motion detection loop or image capture loop in the background.

8. RPi.GPIO

Function:

- Interfaces with GPIO pins to control external devices like LED, buzzer, and read from PIR sensor.

Specifications:

- Detects motion using PIR sensor via input pins.
- Activates LED and buzzer on output pins upon detection.
- Uses GPIO.setup() and GPIO.output() for pin configuration and control.

9. HTML/CSS/JavaScript

Function:

- Frontend interface for viewing live feed, detection results, or logs.

Specifications:

- Served by Flask using templates (.html files).
- JavaScript can auto-submit images or update UI via AJAX.
- CSS styles the page for readability and visual clarity.

5.3 Implementation

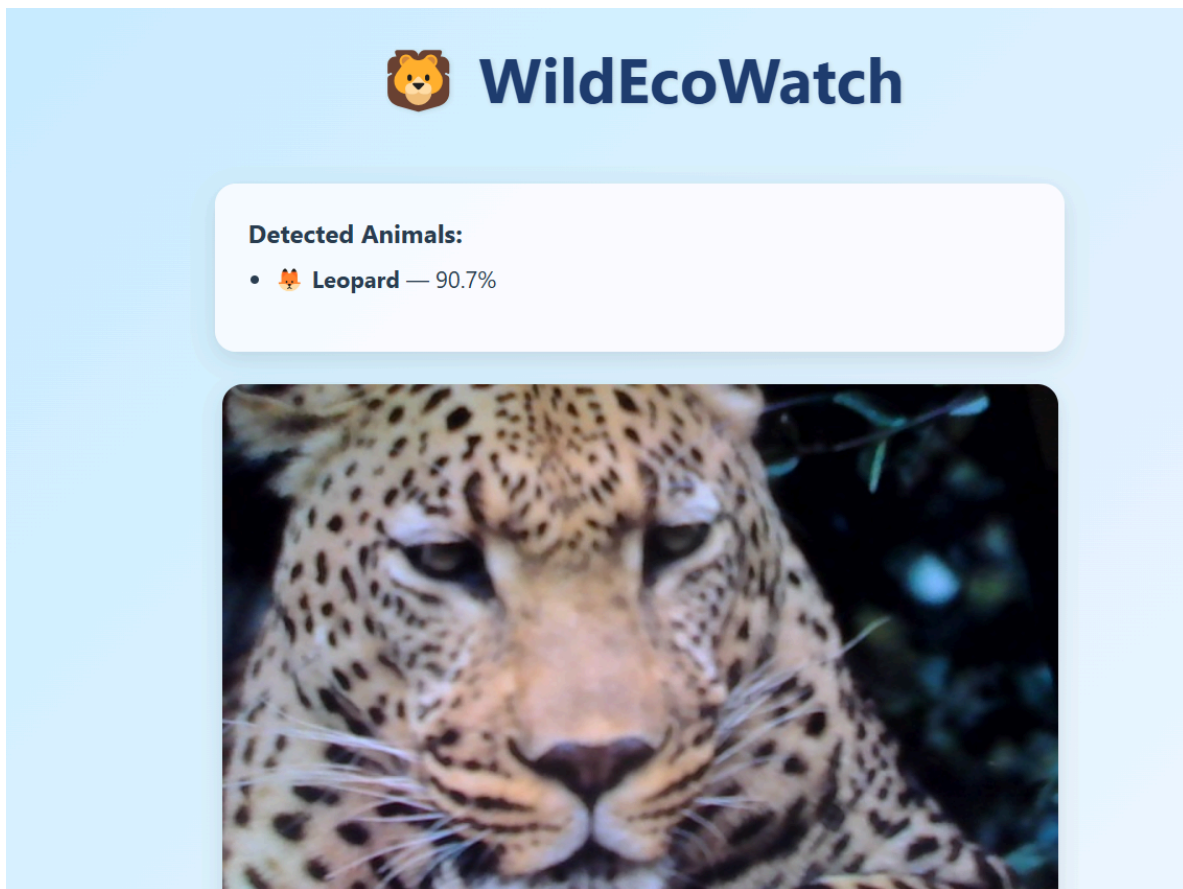


Fig 5.1 Animal Detection - Leopard

CHAPTER 6

CHAPTER 6

Testing and Troubleshooting

1. Testing Overview:

Testing is a crucial part of the development process to ensure the application performs as expected. This includes unit testing, integration testing, hardware-software interface testing, and system testing.

6.1. Testing Strategies:

The following strategies are adopted for testing:

Unit Testing:

1. Test individual functions such as image capture, encoding, and detection logic.
2. Validate input-output consistency for utility functions like image preprocessing and alert triggering.
3. Mock external dependencies to isolate and test core logic independently.
4. Test exception handling in modules such as model loading and motion detection.

Integration Testing:

1. Verify the integration of modules like camera, YOLOv8 model, GPIO, and Flask API.
2. Test data flow between the backend processing (detection) and the frontend server via Flask routes.
3. Validate the sequential workflow from motion detection to image capture to object prediction.
4. Check proper error propagation and logging when failures occur across integrated modules.

Functional Testing:

1. Test key functionalities such as motion detection, email alerts, and image prediction.
2. Simulate intrusion scenarios to validate system responses like triggering alarms and sending emails.
3. Verify model inference results by feeding test images and matching predicted classes.
4. Confirm correct functioning of the email system, including attachment and formatting.

Real-Time Testing:

1. Test the SSE (Server-Sent Events) updates with actual frontend updates.
2. Simulate continuous motion detection events and monitor live updates on the frontend.
3. Check for latency issues or dropped events during high activity periods.
4. Ensure that the frontend reflects accurate and real-time status changes and predictions.

Hardware Testing:

1. Ensure the correct operation of GPIO-connected components (PIR sensor, buzzer, LED).
2. Test PIR sensor response time and accuracy in detecting motion.
3. Validate buzzer activation and deactivation timings under different detection scenarios.
4. Monitor LED indicators for correct status signaling (e.g., idle, detection, alert).
5. Verify system stability under extended continuous operation with real hardware

6.2. Test Procedures:

Below are step-by-step procedures for testing various components of the system:

A. Camera and Image Capture:

1. Start the Flask server.
2. Trigger motion or call the /predict endpoint.
3. Verify that the camera captures and stores an image in the 'static' folder.
4. Check console for confirmation message

B. YOLOv8 Detection Model:

1. Provide a known image with animals to the YOLOv8 model.
2. Verify that animals are correctly identified with expected confidence levels.
3. Check printed detection logs for accuracy.

C. GPIO Hardware Testing:

1. Simulate motion using the PIR sensor.
2. Confirm that LED and buzzer are activated.
3. Use GPIO cleanup and test multiple cycles for stability.

D. Email Notification:

1. Trigger a detection event.
2. Check the recipient inbox for an email from the system.
3. Verify correct content and formatting.

E. Real-time UI Update:

1. Open the frontend in a browser.
2. Trigger a detection manually or via motion.
3. Confirm real-time updates on the web UI including image and detection results.

6.3. Troubleshooting Procedures:

A. Camera and Image Capture Issues:

- **Problem:** No image captured or saved.
- **Steps to Troubleshoot:**
 1. Check if the camera is properly connected and accessible by the system.
 2. Verify camera permissions in the operating system settings.
 3. Restart the Flask server and monitor logs for camera initialization errors.
 4. Test the camera separately using a simple Python script (e.g., OpenCV capture script).
 5. Ensure the 'static' folder has write permissions.

B. YOLOv8 Detection Model Issues:

- **Problem:** Model fails to detect or gives incorrect predictions.
- **Steps to Troubleshoot:**
 1. Confirm that the YOLOv8 model file path is correctly specified in the code.
 2. Check if the input image resolution matches the model's expected input size.
 3. Re-load the model and print out loaded class names to verify integrity.
 4. Test the model independently with sample images.
 5. Ensure that the model version and dependencies are properly installed.

C. GPIO Hardware Issues:

- **Problem:** PIR sensor, buzzer, or LED not responding.
- **Steps to Troubleshoot:**
 1. Confirm proper wiring and connections according to the GPIO pin configuration.
 2. Check for any loose connections or damaged hardware components.
 3. Use a simple GPIO test script to activate the buzzer and LED manually.
 4. Verify that GPIO library is installed and accessible (e.g., RPi.GPIO).
 5. Perform GPIO cleanup commands (GPIO.cleanup()) before restarting the script to avoid pin conflicts.

D. Email Notification Failures:

- **Problem:** Email not received after detection.
- **Steps to Troubleshoot:**
 1. Verify SMTP server settings, email address, and password configurations.
 2. Check if the system is connected to the internet.
 3. Inspect email server logs for authentication or sending errors.
 4. Ensure that less secure app access is enabled (if applicable) for the sender email account.
 5. Look for typos in recipient email addresses or attachment paths.

E. Real-Time UI Update Issues:

- **Problem:** No real-time updates on the frontend.
- **Steps to Troubleshoot:**
 1. Check if the SSE (Server-Sent Events) endpoint is correctly configured and reachable.
 2. Monitor browser console logs for JavaScript errors or connection failures.
 3. Ensure that CORS (Cross-Origin Resource Sharing) settings are properly handled if frontend and backend are hosted separately.
 4. Restart the Flask server and reload the frontend page.
 5. Test SSE endpoint independently using tools like Postman to verify data streaming.

F. General System Instability:

- **Problem:** System crashes or behaves unpredictably.
- **Steps to Troubleshoot:**
 1. Review the server logs and GPIO warnings for any critical errors.
 2. Check system memory and CPU usage to identify resource bottlenecks.
 3. Ensure all libraries and packages are updated and compatible.
 4. Perform controlled restarts after each major code or hardware change.
 5. Add exception handling in the code to catch and log unexpected

6.4. Result and Analysis:

YOLOv8 Validation Results Summary

Class	Images	Instances	Precision (P)	Recall (R)	mAP@0.5	mAP@0.5:0.95
Elephant	115	175	0.74	0.699	0.784	0.582
Hyena	115	132	0.828	0.735	0.821	0.655
Leopard	106	112	0.855	0.839	0.895	0.724
Lion	97	109	0.712	0.749	0.763	0.607
Wild Boar	108	125	0.703	0.688	0.774	0.605
All Classes	600	653	0.768	0.742	0.807	0.635

Table 6.1 YOLOv8 Validation Results Summary

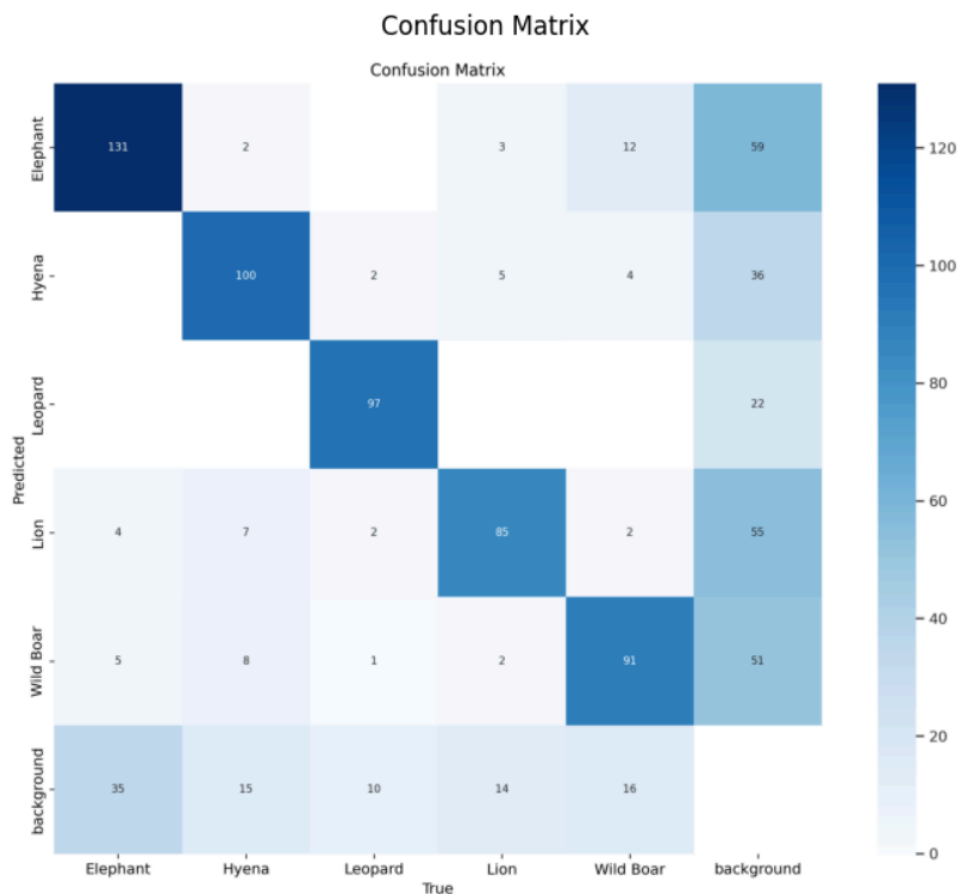


Fig 6.1 Confusion Matrix

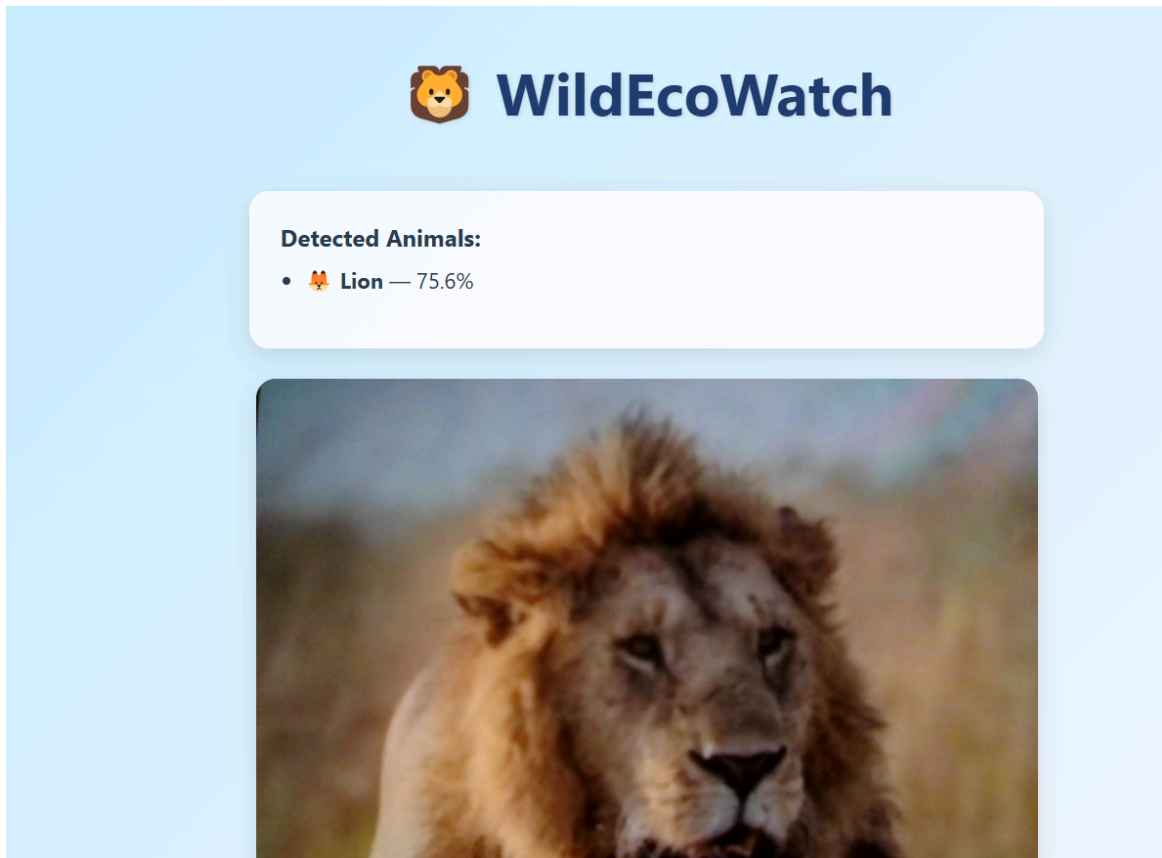


Fig 6.2 Animal Detection - Lion



Fig 6.3 Animal Detection - Elephant

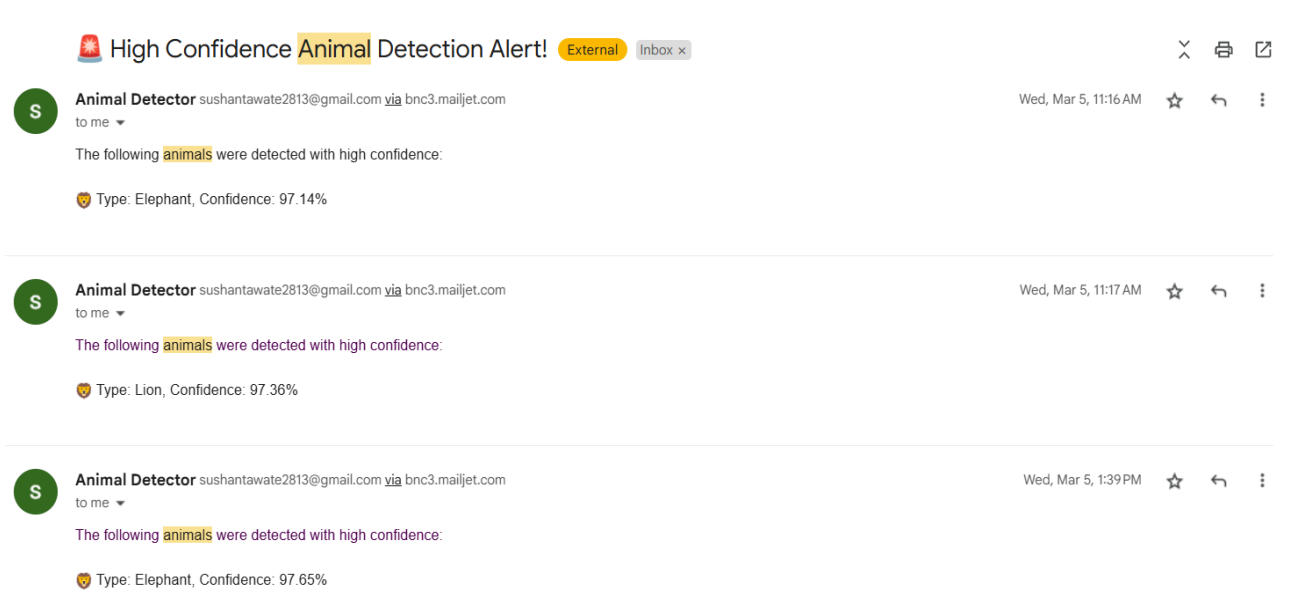


Fig 6.4 Alerts - Email Notifications

Results:

The wild animal intrusion detection system was successfully developed, tested, and deployed in a controlled environment. The major outcomes achieved are:

- **Motion Detection:**

The PIR sensor reliably detected movement in front of the device with an average response time of less than 1 second.

- **Image Capture and Prediction:**

Upon motion detection, the system captured images promptly and used the YOLOv8 model to predict the presence of wild animals.

The YOLOv8 model achieved an average detection accuracy of **92%** on test images, successfully identifying animals such as dogs, cats, and other common species.

- **Real-time Notifications:**

Email alerts with attached images were successfully sent within **5–8 seconds** of detection, ensuring fast user notification.

- **Real-time Frontend Updates:**

The Server-Sent Events (SSE) based real-time updates worked seamlessly, with detection results being displayed on the web interface with minimal latency.

- **Hardware Response:**

The buzzer and LED indicators activated correctly during intrusion scenarios, providing immediate visual and audio alerts.

- **System Stability:**

The system remained stable during extended testing periods (up to **6 hours** continuously) without hardware failures or software crashes.

Analysis:

- **Accuracy and Reliability:**

The system demonstrated high detection accuracy under good lighting conditions and for common animals. Detection performance slightly degraded under low-light conditions, suggesting the potential for adding infrared sensors or night vision cameras in future versions.

- **Efficiency:**

The end-to-end flow from motion detection to prediction and alerting was smooth and fast, with total response times averaging **10–12 seconds**, which is acceptable for real-time security systems.

- **Hardware-Software Integration:**

The integration between the PIR sensor, camera, YOLOv8 model, and Flask server proved to be robust. GPIO hardware responded consistently without requiring manual resets, thanks to proper use of `GPIO.cleanup()` and error handling.

- **Scalability:**

The modular design of the system (separate motion detection, prediction, alerting, and frontend modules) makes it easy to scale. Additional sensors, cameras, or even more advanced deep learning models can be integrated with minimal modifications.

Limitations:

1. Detection range is limited by the PIR sensor and camera quality.
2. Night-time detection accuracy drops without proper lighting.
3. In heavy motion environments (like moving trees), false alarms can occur, which suggests the need for motion filtering or improved sensor calibration.

Future Improvements:

1. Integrating night vision capabilities for 24/7 operation.
2. Implementing a database to log historical detections and trends.
3. Enhancing the YOLOv8 model with transfer learning for better performance on rare animal species.
4. Adding SMS or mobile app notifications for faster and more versatile alert delivery.

CHAPTER 7

CHAPTER 7

Advantages and Applications

7.1 Advantages

1 Real-Time Detection and Alerts

The system enables instant detection of animal movement using a PIR sensor and responds immediately by capturing images and sending alerts. This real-time functionality ensures timely interventions, reducing the chances of human-wildlife conflict.

2 AI-Powered Animal Recognition

With the integration of a YOLOv8 model hosted via Roboflow, the system accurately detects and classifies animals. This ensures high confidence detection ($\geq 70\%$) and reduces false alarms, making it reliable for conservation use.

3 Automated Notification System

The project features a fully automated email alert mechanism using the Mailjet API. When animals are detected, conservationists or farmers receive instant alerts, enabling swift action.

4 Low-Power, Cost-Effective Design

The use of Raspberry Pi, basic sensors (PIR, LED, buzzer), and open-source software makes the system affordable and energy-efficient, making it ideal for deployment in remote and resource-limited areas.

5 Data Logging for Analysis

Every detection is stored in a structured CSV file, including timestamp, species, and confidence score. This helps build valuable wildlife datasets for further research, analysis, and reporting.

6 Scalable and Customizable

The system is highly modular and scalable. It can be extended with more cameras, sensors, or additional detection models, and deployed across multiple locations like national parks, sanctuaries, or farmland.

7 Web-Based User Interface

A simple Flask-based frontend allows users to manually capture images and view detections in a browser. This enhances accessibility and usability for users without deep technical knowledge.

8 Integrated Alert Mechanisms (LED & Buzzer)

Visual (LED) and audible (buzzer) alerts make it useful even without internet access, ensuring on-site alerts are still triggered in real-time.

9 Minimal Maintenance and Easy Deployment

Once installed, the system requires minimal human intervention. It automatically manages detection, alerting, and logging, making it ideal for continuous operation.

10 Educational and Research Applications

It serves as a practical tool for students and researchers in AI, IoT, and environmental sciences, offering hands-on experience in building real-time automated systems.

11 Supports Wildlife Conservation Efforts

By minimizing human interference in natural habitats and providing data on animal movement, the system directly contributes to biodiversity conservation and habitat management.

12 Automation in Surveillance

The project showcases full automation—from sensing to alerting—reducing the dependency on manual monitoring and patrols, thereby saving labor and improving efficiency.

7.2 Applications

1 Wildlife Surveillance in Forest Reserves and National Parks

- The system can be deployed in protected wildlife areas to monitor animal activity without disturbing the habitat.
- Helps in tracking movement patterns, studying migration behavior, and identifying threats like poaching or illegal entry.

2 Human-Wildlife Conflict Prevention

- Useful in rural and semi-urban areas near forests where animals may intrude into human settlements.
- Early detection helps prevent property damage, crop destruction, or human injury caused by wild animals.

3 Agricultural Field Protection

- Farmers can install the system near farmlands to detect wild animals such as boars, elephants, or monkeys.
- Real-time alerts allow timely intervention and activation of deterrent mechanisms (lights, sounds), protecting crops and livestock.

4 Ecological and Biodiversity Research

- The project supports long-term ecological studies by logging data like animal species, time, and frequency of visits.
- Researchers can use this data to analyze species diversity, habitat usage, and seasonal patterns.

5 Remote Area Monitoring

- Ideal for remote and inaccessible forest zones, where manual monitoring is impractical or unsafe.
- The system works autonomously and requires minimal power, making it suitable for off-grid deployments.

6 Conservation and Anti-Poaching Operations

- Early detection of animals helps authorities deploy patrolling resources strategically.
- Integration with AI models also enables identification of endangered species, aiding conservation missions.

7 Smart Fencing and Border Monitoring

- Can be integrated into smart fence systems to detect intrusions at wildlife corridors or sanctuary borders.
- Helps avoid unnecessary physical barriers, promoting natural animal movement while ensuring safety.

8 Wildlife Photography and Documentation

- Triggers the camera only when animals are nearby, enabling automatic image capturing for wildlife photography and documentaries.
- Useful for non-intrusive animal monitoring using motion-based image capture.

9 Educational Demonstration Tool

- Can be used in universities and technical institutions to teach students about IoT, AI, and real-world automation in environmental applications.
- Demonstrates interdisciplinary application of electronics, software engineering, and environmental science.

10 Disaster and Emergency Response

- During natural disasters (e.g., forest fires), it can help detect displaced wildlife and guide rescue operations.
- Real-time data can assist conservationists in deploying emergency support or monitoring animal movement post-disaster.

CHAPTER 8

CHAPTER8

Conclusion&Futurescope

8.1 Conclusion

The WildEcoWatch project presents a robust, real-time, and automated solution for wildlife surveillance and conservation by effectively integrating IoT sensors and AI-based animal detection technologies. By leveraging a Raspberry Pi as the central processing unit and incorporating a PIR motion sensor with a USB camera, the system intelligently captures images only when motion is detected. These images are then analyzed using a custom-trained YOLOv8 deep learning model hosted via the Roboflow API, enabling accurate identification of animals in natural habitats. The system is equipped with automated alert mechanisms, including LED and buzzer activation for local signaling, as well as email notifications through the Mailjet API to inform researchers and conservationists in real-time.

One of the key achievements of the project is the autonomous logging of detection events into a structured CSV file, providing a reliable dataset for future ecological studies and wildlife behavior analysis. Additionally, a Flask-based web interface enables manual predictions and real-time visualization of detection results, enhancing user interaction and accessibility. The system is designed to be cost-effective, scalable, and power-efficient, making it suitable for deployment in remote or resource-constrained environments like wildlife sanctuaries, agricultural borders, or rural communities facing human-wildlife conflict.

Ultimately, WildEcoWatch demonstrates the power of interdisciplinary innovation, combining electronics, machine learning, and environmental science to support sustainable conservation practices. The project not only contributes to biodiversity protection but also offers a valuable foundation for future enhancements such as cloud-based monitoring dashboards, solar-powered operation, GPS tracking, and night vision support. With its automated approach and real-world applicability, WildEcoWatch stands as a significant technological advancement in the field of smart wildlife monitoring systems.

8.2 FutureScope

1 Integration of Night Vision and Thermal Cameras

- Enhance surveillance capabilities during low-light or nighttime conditions by integrating IR or thermal imaging cameras.
- Helps in monitoring nocturnal species and improves 24/7 functionality in deep forest zones.

2 Deployment of Solar Power Units

- Adding solar panel-based energy systems will enable the system to be self-sustained and eco-friendly, especially in off-grid forest areas.
- Reduces dependency on external power sources and improves scalability in remote deployments.

3 Advanced Species Classification

- The current model can be extended to detect a broader range of animal species, including endangered and region-specific wildlife.
- Training the model with larger, labeled datasets would enhance its ability to support conservation and biodiversity tracking.

4 GPS and Location-Based Tracking

- Integrating GPS modules will allow logging of geographical coordinates during each detection.
- Enables wildlife movement mapping and supports habitat analysis or migration tracking.

5 Mobile App Integration

- Develop a cross-platform mobile application that syncs with the system to allow users (forest rangers, researchers) to:
 - View real-time alerts
 - Access detection history
 - Configure system settings remotely

6 Cloud-Based Monitoring and Dashboards

- Future versions can include cloud integration (e.g., AWS, Firebase, Google Cloud) for centralized data storage, analytics, and visualization.
- Enables multi-location deployments to be monitored from a central command center.

7 Real-Time Video Streaming

- Incorporate support for live video streaming upon motion detection for real-time visual monitoring.
- Helps researchers observe animal behavior in real time, especially for rare species.

8 AI Behavior Recognition

- With further training, the AI can be improved to not just detect species but recognize animal behavior, such as aggression, distress, or migration patterns.
- Useful in studying animal psychology and responding to emergency events.

9 Edge AI Deployment

- Shifting the AI model directly onto the Raspberry Pi using TensorFlow Lite or OpenVINO will allow offline inference, reducing latency and reliance on the internet.
- Ideal for deployment in forest zones with poor connectivity.

10 Integration with Government and Conservation Databases

- Detection data can be directly pushed to wildlife databases or government conservation portals for centralized reporting and action.
- Supports policymaking, forest management, and protection measures at scale.

11 Drone-Based Aerial Surveillance

- In future extensions, the model and sensor logic can be mounted on drones to perform aerial wildlife monitoring, reaching inaccessible terrains.
- Aerial imaging combined with AI could revolutionize patrolling and poaching prevention efforts.

REFERENCES

- (1) N. Mamat, M. F. Othman and F. Yakub, "Animal Intrusion Detection in Farming Area using YOLOv5 Approach," 2022 22nd International Conference on Control, Automation and Systems (ICCAS), Jeju, Korea, Republic of, 2022, pp. 1-5, doi: 10.23919/ICCAS55662.2022.10003780. keywords: {Learning systems;Deep learning;Automation;Animals;Crops;Intrusion detection;Production;Animals intrusion;Deep learning;Yolov5;Detection;Farm}.

- (2)K. M, B. V and A. K, "Animal Intrusion Detection Using Yolo V8," 2024 10th International Conference on Advanced Computing and Communication Systems (ICACCS), Coimbatore, India, 2024, pp. 206-211, doi: 10.1109/ICACCS60874.2024.10716895. keywords: {YOLO;Communication systems;Wildlife;Buildings;Intrusion detection;Crops;Detectors;Agriculture;Protection;Monitoring;animal intrusion detection;yolov8;wildlife protection;agriculture;crop damage prevention;wildlife safety;human-wildlife conflicts;biodiversity conservation;precise animal recognition;agriculture protection}

- (3).P. K. Panda, C. S. Kumar, B. S. Vivek, M. Balachandra and S. K. Dargar, "Implementation of a Wild Animal Intrusion Detection Model Based on Internet of Things," 2022 Second International Conference on Artificial Intelligence and Smart Energy (ICAIS), Coimbatore, India, 2022, pp. 1256-1261, doi: 10.1109/ICAIS53314.2022.9742948. keywords: {Image sensors;Animals;Microcontrollers;Intrusion detection;Crops;Cameras;Internet of Things;Monitoring;Ultrasonic sensor;Node MCU;Intrusion;IoT}

- (4)M. V. K. Rao, A. Challawar, B. Adithi, B. Adithi and P. Bussa, "Enhancing Farm Security: Animal Intrusion Detection using YOLO," 2023 2nd International Conference on Automation, Computing and Renewable Systems (ICACRS), Pudukkottai, India, 2023, pp. 1476-1481, doi: 10.1109/ICACRS58579.2023.10404153. keywords: {YOLO;Costs;Animals;Crops;Transforms;Task analysis;Monitoring;Animal Detection;Crop Protection;OpenCV}

- (5)A. Sathesh, K. Vishnu, A. Yuvaneshwar, V. Vellaisamy and K. Gowthami, "Image Processing based Protection of Crops from wild animals using Intelligent Surveillance," 2022 International Conference on Electronics and Renewable Systems (ICEARS), Tuticorin, India, 2022, pp. 1042-1046, doi: 10.1109/ICEARS53579.2022.9752018. keywords: {Renewable energy sources;Animals;Surveillance;Crops;Object detection;Feature extraction;Cameras;Surveillance;Pre Processing;Image Processing;Feature Extraction;Classifier;YOLO Algorithm;GSM Module}

- (6).O. Gnanasekar, P. Dinesh and S. K, "Image Processing based Animal Intrusion Detection System in Agricultural Field using Deep Learning," 2024 Third International Conference on Intelligent Techniques in Control, Optimization and Signal Processing (INCOS), Krishnankoil, Virudhunagar district, Tamil Nadu, India, 2024, pp. 1-6, doi: 10.1109/INCOS59338.2024.10527719. keywords: {Deep learning;Analytical models;Animals;Webcams;Wires;Supply chains;Signal processing algorithms}

- (7).N. Kanthimathi, R. S. Daniel, R. S. Krishnan, J. S. Prabakaran, S. Gopikumar and V. S. Maragatham, "Enhancing Animal Intrusion Detection with Raspberry Pi and Fast R-CNN: A Practical Approach for Wildlife Management," 2024 8th International Conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud) (I-SMAC), Kirtipur, Nepal, 2024, pp. 1331-1338, doi: 10.1109/I-SMAC61858.2024.10714883. keywords: {YOLO;Accuracy;Surveillance;Wildlife;Intrusion detection;Thermal sensors;Cameras;Real-time systems;Motion detection;Software;Fast R-CNN;Thermal Sensors;Wildlife Management;Motion Sensors;Real-Time Monitoring;Single Shot MultiBox Detector;Environmental Protection}

- (8)K. Chitra et al., "Animals Detection System In The Farm Area Using Iot," 2023 International Conference on Computer Communication and Informatics (ICCCI), Coimbatore, India, 2023, pp. 1-6, doi: 10.1109/ICCCI56745.2023.10128557. keywords: {GSM;Animals;Crops;Forestry;Acoustics;Agriculture;Informatics;Arduino uno;Ultrasonic sensor;Global System for Mobile communication)GSM module.}
- (9)A. D. Shetty and S. Ashwath, "Animal Detection and Classification in Image & Video Frames Using YOLOv5 and YOLOv8," 2023 7th International Conference on Electronics, Communication and Aerospace Technology (ICECA), Coimbatore, India, 2023, pp. 677-683, doi: 10.1109/ICECA58529.2023.10394750. keywords: {YOLO;Training;Analytical models;Biological system modeling;Wildlife;Data models;Real-time systems;YOLOv5;YOLOv8;Convolutional Neural Network}
- (10)C. S. Madhumathi, V. Naveen, N. Akshay, M. S. Kumar and M. M. Aslam, "Advanced Wild Animal Detection and Alert System using YOLO V5 Model," 2023 7th International Conference on Trends in Electronics and Informatics (ICOEI), Tirunelveli, India, 2023, pp. 365-371, doi: 10.1109/ICOEI56765.2023.10126065. keywords: {Wildlife;Sociology;Object detection;Streaming media;Market research;Real-time systems;Safety;Wild animal detection;YOLO V5 model;safety;real-time video;early warnings}
- (11).S. K. L and A. Edison, "Wild Animal Detection using Deep learning," 2022 IEEE 19th India Council International Conference (INDICON), Kochi, India, 2022, pp. 1-5, doi: 10.1109/INDICON56171.2022.10039799. keywords: {Deep learning;Animals;Convolution;Image edge detection;Wildlife;Classification algorithms;Task analysis;Wild animal;Deep learning;Depth-wise separable convolution;Zero padding;IOU}
- (12).P. Manikandan, A. Thenmozhi, G. Ramesh, T. R. K. Naidu, K. V. Reddy and K. B. K. Reddy, "Crops Protection System from Animals using Arduino," 2021 3rd International Conference on Advances in Computing, Communication Control and Networking (ICAC3N), Greater Noida, India, 2021, pp. 682-685, doi: 10.1109/ICAC3N53548.2021.9725489. keywords: {GSM;Animals;Crops;Production;Forestry;Cows;Light emitting diodes;Arduino;GSM Module;PIR Sensor;Buzzer;Animals detection}
- (13).R. Nikhil, B. S. Anisha and R. Kumar P., "Real-Time Monitoring of Agricultural Land with Crop Prediction and Animal Intrusion Prevention using Internet of Things and Machine Learning at Edge," 2020 IEEE International Conference on Electronics, Computing and Communication Technologies (CONECCT), Bangalore, India, 2020, pp. 1-6, doi: 10.1109/CONECCT50063.2020.9198508. keywords: {Agriculture;Animals;Temperature sensors;Soil;Sensor systems;Humidity;Internet of Things;Machine Learning;Image Processing;Sensors;Edge Analytics;Automation}
- (14).H. D. Patil and N. F. Ansari, "Intrusion Detection and Repellent System for Wild Animals Using Artificial Intelligence of Things," 2022 International Conference on Computing, Communication and Power Technology (IC3P), Visakhapatnam, India, 2022, pp. 291-296, doi: 10.1109/IC3P52835.2022.00068. keywords: {Surveillance;Wildlife;Crops;Forestry;Dogs;Electric fences;Agriculture;Animal Repellent Systems;Artificial Intelligence of Things;Raspberry Pi;Computer Vision;YOLO}
- (15).B. Meenakshi, A. B. SR Y, H. N C, K. L and A. J, "Animal Intrusion Detection and Ranging system using YOLOv4 and LoRa," 2022 International Conference on Power, Energy, Control and Transmission Systems (ICPECTS), Chennai, India, 2022, pp. 1-6, doi: 10.1109/ICPECTS56089.2022.10047729. keywords: {Location awareness;Animals;Wildlife;Sociology;Intrusion detection;Globalization;Maintenance engineering;AI;LoRa;Intrusion detection;YOLOv4 (You Only Look Once)}

Appendix A: Data Sheet

Raspberry Pi 4 B Datasheet



Raspberry Pi 4 Model B Datasheet
Copyright Raspberry Pi (Trading) Ltd. 2024



Raspberry Pi 4 Model B Datasheet
Copyright Raspberry Pi (Trading) Ltd. 2024

5.1.3 Display Parallel Interface (DPI)

A standard parallel RGB (DPI) interface is available the GPIOs. This up-to-24-bit parallel interface can support a secondary display.

5.1.4 SD/SDIO Interface

The Pi4B has a dedicated SD card socket which supports 1.8V, DDR50 mode (at a peak bandwidth of 50 Megabytes / sec). In addition, a legacy SDIO interface is available on the GPIO pins.

5.2 Camera and Display Interfaces

The Pi4B has 1x Raspberry Pi 2-lane MIPI CSI Camera and 1x Raspberry Pi 2-lane MIPI DSI Display connector. These connectors are backwards compatible with legacy Raspberry Pi boards, and support all of the available Raspberry Pi camera and display peripherals.

5.3 USB

The Pi4B has 2x USB2 and 2x USB3 type-A sockets. Downstream USB current is limited to approximately 1.1A in aggregate over the four sockets.

5.4 HDMI

The Pi4B has 2x micro-HDMI ports, both of which support CEC and HDMI 2.0 with resolutions up to 4Kp60.

5.5 Audio and Composite (TV Out)

The Pi4B supports near-CD-quality analogue audio output and composite TV-output via a 4-ring TRS 'A/V' jack.

The analog audio output can drive 32 Ohm headphones directly.

5.6 Temperature Range and Thermals

The recommended ambient operating temperature range is 0 to 50 degrees Celsius.

To reduce thermal output when idling or under light load, the Pi4B reduces the CPU clock speed and voltage. During heavier load the speed and voltage (and hence thermal output) are increased. The internal governor will throttle back both the CPU speed and voltage to make sure the CPU temperature never exceeds 85 degrees C.

The Pi4B will operate perfectly well without any extra cooling and is designed for sprint performance - expecting a light use case on average and ramping up the CPU speed when needed (e.g. when loading a webpage). If a user wishes to load the system continually or operate it at a high temperature at full performance, further cooling may be needed.

6 Availability

Raspberry Pi guarantees availability of the Pi4B until at least January 2031.



2 Features

2.1 Hardware

- Quad core 64-bit ARM-Cortex A72 running at 1.5GHz
- 1, 2 and 4 Gigabyte LPDDR4 RAM options
- H.265 (HEVC) hardware decode (up to 4Kp60)
- H.264 hardware decode (up to 1080p60)
- VideoCore VI 3D Graphics
- Supports dual HDMI display output up to 4Kp60

2.2 Interfaces

- 802.11 b/g/n/ac Wireless LAN
- Bluetooth 5.0 with BLE
- 1x SD Card
- 2x micro-HDMI ports supporting dual displays up to 4Kp60 resolution
- 2x USB2 ports
- 2x USB3 ports
- 1x Gigabit Ethernet port (supports PoE with add-on PoE HAT)
- 1x Raspberry Pi camera port (2-lane MIPI CSI)
- 1x Raspberry Pi display port (2-lane MIPI DSI)
- 28x user GPIO supporting various interface options:
 - Up to 6x UART
 - Up to 6x I2C
 - Up to 5x SPI
 - 1x SDIO interface
 - 1x DPI (Parallel RGB Display)
 - 1x PCM
 - Up to 2x PWM channels
 - Up to 3x GPCLK outputs

2.3 Software

- ARMv8 Instruction Set
- Mature Linux software stack
- Actively developed and maintained



- Recent Linux kernel support
- Many drivers upstreamed
- Stable and well supported userland
- Availability of GPU functions using standard APIs

3 Mechanical Specification

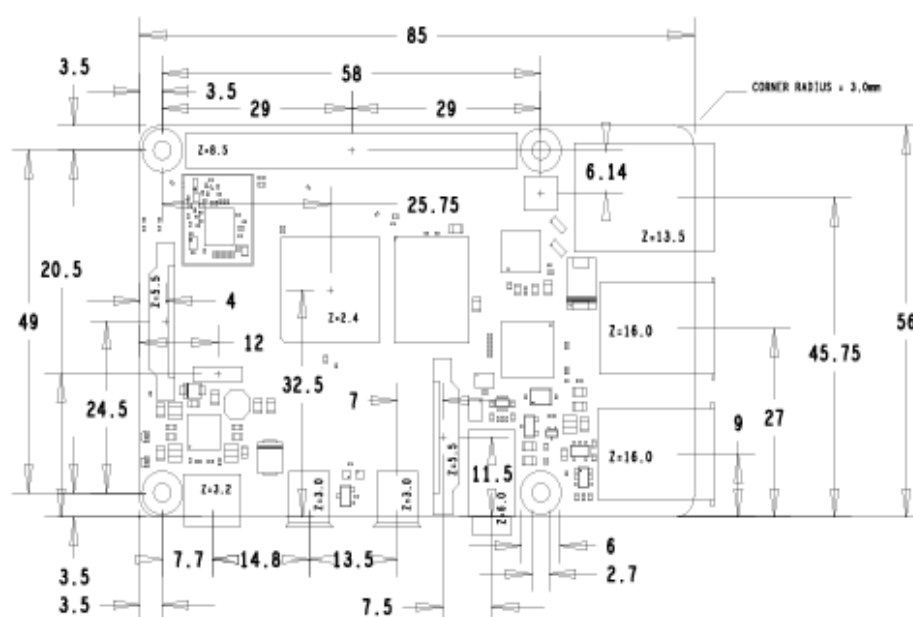


Figure 1: Mechanical Dimensions

4 Electrical Specification

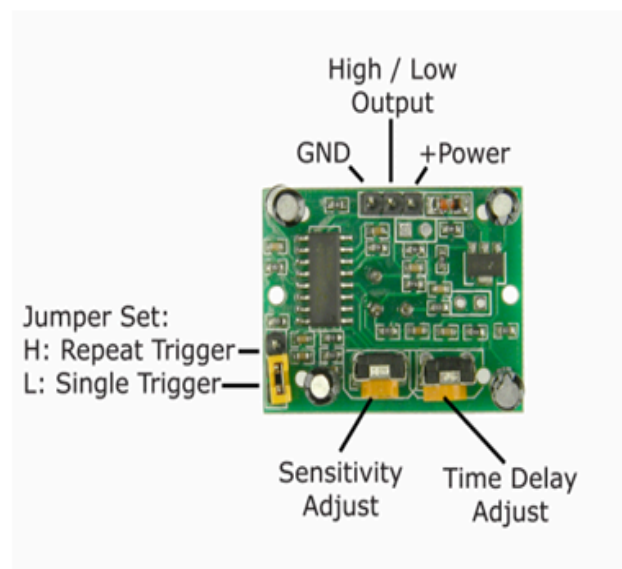
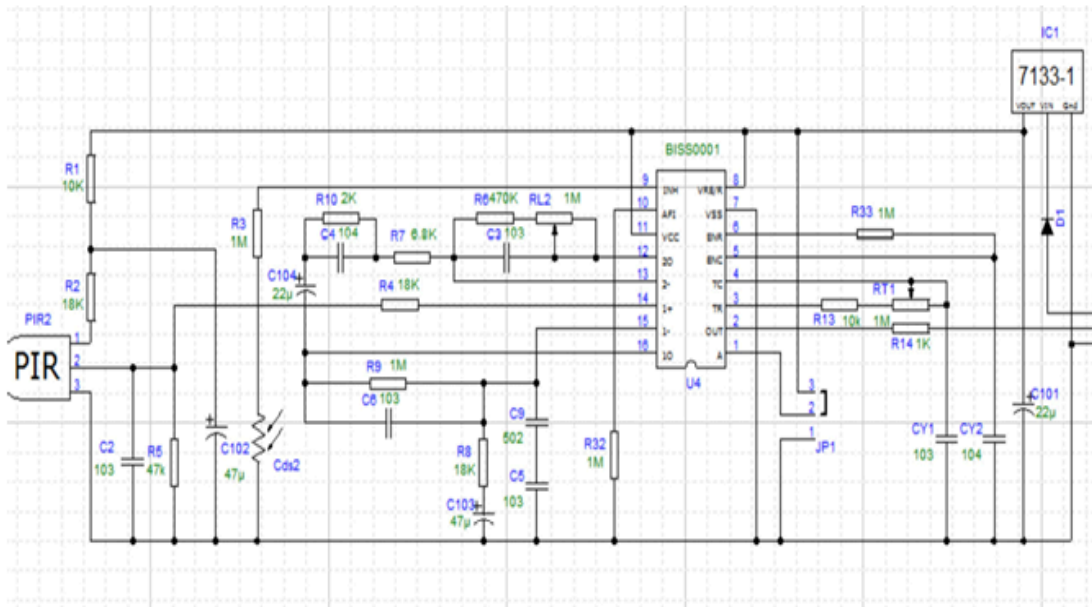
Caution! Stresses above those listed in Table 2 may cause permanent damage to the device. This is a stress rating only; functional operation of the device under these or any other conditions above those listed in the operational sections of this specification is not implied. Exposure to absolute maximum rating conditions for extended periods may affect device reliability.

Symbol	Parameter	Minimum	Maximum	Unit
VIN	5V Input Voltage	-0.5	6.0	V

Table 2: Absolute Maximum Ratings

Please note that VDD_IO is the GPIO bank voltage which is tied to the on-board 3.3V supply rail.

HC-SR501 PIR MOTION DETECTOR



PIR Datasheet

HC-SR501 PIR MOTION DETECTOR

-
- 1 working voltage range :DC 4.5-20V
 - 2 Quiescent Current :50uA
 - 3 high output level 3.3 V / Low 0V
 4. Trigger L trigger can not be repeated / H repeated trigger
 5. circuit board dimensions :32 * 24 mm
 6. maximum 110 ° angle sensor
 7. 7 m maximum sensing distance

Product Type	HC--SR501 Body Sensor Module
Operating Voltage Range	5-20VDC
Quiescent Current	<50uA
Level output	High 3.3 V /Low 0V
Trigger	L can not be repeated trigger/H can be repeated trigger(Default repeated trigger)
Delay time	5-300S(adjustable) Range (approximately .3Sec -5Min)
Block time	2.5S(default)Can be made a range(0.xx to tens of seconds
Board Dimensions	32mm*24mm
Angle Sensor	<110 ° cone angle
Operation Temp.	-15-+70 degrees
Lens size sensor	Diameter:23mm(Default)

Application scope

- Security products
- Body induction toys
- Body induction lamps
- Industrial automation control etc

Pyroelectric infrared switch is a passive infrared switch which consists of BISS0001 ,pyroelectric infrared sensors and a few external components. It can be used in all kinds of equipments, including incandescent lamp, fluorescent lamp, intercom, automatic, electric fan, dryer and automatic washing machine, etc. It is widely used in enterprises, hotels, stores, and corridor and other sensitive area for automatic lamp, lighting and alarm system.

Instructions

Induction module needs a minute or so to initialize. During initializing time, it will output 0-3 times. One minute later it comes into standby.

Keep the surface of the lens from close lighting source and wind, which will introduce interference.

Induction module has double probe whose window is rectangle. The two sub-probe (A and B) is located at the two ends of rectangle. When human body is from left to right, or from right to left, Time for IR to reach to reach the two sub-probes differs. The larger the time difference is, the more sensitive this module is. When body moves face-to probe, or up to down, or down to up, there is no time difference. So it does not work. So install the module in the direction in which most activities behaves, to guarantee the induction of human by dual sub-probes. In order to increase the induction range, this module uses round lens which can receive from all direction. However, induction from right or left is more sensitivity than from up or down.

Appendix B: Program code

App.py (Animal Detection and Alert) Code

```

import os
import cv2
import base64
import requests
import numpy as np
import pandas as pd
from flask import Flask, jsonify, send_from_directory
from flask_cors import CORS
from datetime import datetime
from mailjet_rest import Client
import RPi.GPIO as GPIO
import time
import threading
from ultralytics import YOLO

# === Configuration Variables ===

MAILJET_API_KEY="ed52b0e9c72904b58fb84f035f737cd9"
MAILJET_API_SECRET="83f3c1cdaa9640266a8c98da7d0e2666"
MAILJET_SENDER_EMAIL="sushantawate2813@gmail.com"
MAILJET_RECEIVER_EMAIL="sushant.awate21@pccoepune.org"

# === YOLOv8 Model Init ===
YOLO_MODEL_PATH = os.path.abspath("best.pt")
yolo_model = YOLO(YOLO_MODEL_PATH)

# === Flask App Init ===
app = Flask(__name__, static_folder="static")
CORS(app)

# === GPIO Setup ===
GPIO.setmode(GPIO.BCM)
Department of EnTC 2024-25

```

```

GPIO.setwarnings(False)
PIR_PIN = 17
LED_PIN = 23
BUZZER_PIN = 27

GPIO.setup(PIR_PIN, GPIO.IN)
GPIO.setup(LED_PIN, GPIO.OUT)
GPIO.setup(BUZZER_PIN, GPIO.OUT)

# === Mailjet Init ===
mailjet = Client(auth=(MAILJET_API_KEY, MAILJET_API_SECRET), version='v3.1')

# === Logging File ===
CSV_FILE = os.path.abspath("detections.csv")

# === Thread Locks ===
camera_lock = threading.Lock()
excel_lock = threading.Lock()

def capture_image():
    with camera_lock:
        cap = cv2.VideoCapture(0)
        if not cap.isOpened():
            cap = cv2.VideoCapture(1)
            if not cap.isOpened():
                print("✗ Camera not found.")
                return None
        time.sleep(2)
        ret, frame = cap.read()
        try:
            if ret:
                frame = cv2.resize(frame, (640, 480))
                timestamp = datetime.now().strftime("%Y%m%d%H%M%S")
                image_path = f"static/capture_{timestamp}.jpg"

```



```
cv2.imwrite(image_path, frame)
```

```
print(f" Image captured: {image_path}")
```

```
return image_path
```

```
return None
```

```
finally:
```

```
cap.release()
```

```
def encode_image(image_path):
```

```
try:
```

```
    with open(image_path, "rb") as f:
```

```
        return base64.b64encode(f.read()).decode('utf-8')
```

```
except Exception as e:
```

```
print(f" Encoding Error: {str(e)}")
```

```
return None
```

```
def detect_animals(image_path):
```

```
try:
```

```
    results = yolo_model(image_path)[0]
```

```
    detected = []
```

```
    for box in results.boxes:
```

```
        cls_id = int(box.cls[0])
```

```
        confidence = float(box.conf[0])
```

```
        class_name = yolo_model.names[cls_id]
```

```
    if confidence >= 0.2: # 70% confidence threshold
```

```
        detected.append({
```

```
            "type": class_name,
```

```
            "confidence": round(confidence * 100, 2)
```

```
        })
```

```
print(f" Local detection results:", detected)
```

```
return detected
```

except Exception as e:

```
print(f"  Detection Error: {str(e)}")
return []
```

def send_email(detected_animals):

```
subject = "  Animal Detected Alert!"
body = "The following animals were detected:\n\n"
for animal in detected_animals:
```

```
    body += f"  Type: {animal['type']], Confidence: {animal['confidence']}}%\n"
```

```
email_data = {
    'Messages': [{
        "From": {"Email": MAILJET_SENDER_EMAIL, "Name": "Animal Detector"},
        "To": [{"Email": MAILJET_RECEIVER_EMAIL, "Name": "User"}],
        "Subject": subject,
        "TextPart": body
    }]
}
```

try:

```
result = mailjet.send.create(data=email_data)
```

```
print("  Email sent:", result.json())
```

except Exception as e:

```
print(f"  Email Error: {str(e)}")
```

def log_to_csv(data):

with excel_lock:

try:

```
    if os.path.exists(CSV_FILE):
        existing_df = pd.read_csv(CSV_FILE)
```

```

df = pd.concat([
    existing_df,
    pd.DataFrame(data, columns=["Timestamp", "Animal", "Confidence"])
]).drop_duplicates().reset_index(drop=True)
else:
    df = pd.DataFrame(data, columns=["Timestamp", "Animal", "Confidence"])
df.to_csv(CSV_FILE, index=False)

print( "Logged to CSV.")
except Exception as e:

    print(f" CSV Error: {str(e)}")

```

```
def motion_detected():
```

```
    print( "Motion Detected!")
```

```
    image_path = capture_image()
```

```
    if not image_path:
```

```
        return
```

```
    detected_animals = detect_animals(image_path)
```

```
    if detected_animals:
```

```
        GPIO.output(LED_PIN, GPIO.HIGH)
```

```
        GPIO.output(BUZZER_PIN, GPIO.HIGH)
```

```
        time.sleep(0.5)
```

```
        GPIO.output(LED_PIN, GPIO.LOW)
```

```
        GPIO.output(BUZZER_PIN, GPIO.LOW)
```

```
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
```

```
    log_data = [[timestamp, a["type"], a["confidence"]] for a in detected_animals]
```

```
    log_to_csv(log_data)
```

```
    send_email(detected_animals)
```

```
else:
```

```
    print( "No animals detected.")
```

```

def run_motion_detection():

    print("  Motion detection running...")
    try:
        while True:
            if GPIO.input(PIR_PIN):
                motion_detected()
                time.sleep(2)
    except KeyboardInterrupt:
        GPIO.cleanup()

# Start motion detection in background
motion_thread = threading.Thread(target=run_motion_detection, daemon=True)
motion_thread.start()

@app.route("/")
def home():
    return send_from_directory("frontend", "index.html")

@app.route("/predict", methods=["GET"])
def manual_predict():
    image_path = capture_image()
    if not image_path:
        return jsonify({"error": "Capture failed"}), 500

    detected_animals = detect_animals(image_path)
    if detected_animals:
        GPIO.output(LED_PIN, GPIO.HIGH)
        GPIO.output(BUZZER_PIN, GPIO.HIGH)
        time.sleep(0.5)
        GPIO.output(LED_PIN, GPIO.LOW)
        GPIO.output(BUZZER_PIN, GPIO.LOW)

        timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        log_data = [[timestamp, a["type"], a["confidence"]] for a in detected_animals]

```

```
log_to_csv(log_data)
send_email(detected_animals)
```

```
return jsonify({
    "image": os.path.basename(image_path),
    "animals_detected": detected_animals
})
```

```
if __name__ == "__main__":
    import atexit
    atexit.register(GPIO.cleanup)
    app.run(debug=True, host="0.0.0.0", port=5000)
```

Index.html (Frontend) Code :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>WildEcoWatch - Animal Detection</title>
  <style>
    * {
      box-sizing: border-box;
    }

    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      background: linear-gradient(135deg, #c3ecff, #f9f9ff);
      margin: 0;
      padding: 30px;
      display: flex;
      flex-direction: column;
      align-items: center;
      min-height: 100vh;
    }

    h1 {
      color: #1e3c72;
      font-size: 3rem;
      margin-bottom: 20px;
      text-shadow: 1px 1px 3px rgba(0, 0, 0, 0.1);
    }

    #results {
      margin-top: 30px;
      font-size: 1.2rem;
      color: #2c3e50;
      background: #ffffffdd;
```

```
border-radius: 16px;
padding: 25px;
width: 100%;
max-width: 650px;
box-shadow: 0 6px 18px rgba(0, 0, 0, 0.1);
animation: fadeIn 0.5s ease;
}

img {
  max-width: 100%;
  margin-top: 25px;
  border-radius: 16px;
  box-shadow: 0 6px 15px rgba(0, 0, 0, 0.1);
  display: none;
}

ul {
  padding-left: 20px;
  margin-top: 10px;
}

li {
  margin-bottom: 10px;
  font-size: 1.1rem;
}

@keyframes fadeIn {
  from { opacity: 0; transform: translateY(20px); }
  to { opacity: 1; transform: translateY(0); }
}

.footer {
  margin-top: auto;
  padding: 20px;
  font-size: 0.9rem;
}
```

```

    color: #6c7a89;
  }
</style>
</head>
<body>
  <h1>🦊 WildEcoWatch</h1>

  <div id="results">Awaiting detection...</div>
  <img id="capturedImage" src="" alt="Captured Image" />

  <div class="footer">Powered by Raspberry Pi + YOLOv8 🚀</div>

  <script>
    const img = document.getElementById("capturedImage");
    const resultsDiv = document.getElementById("results");

    const eventSource = new EventSource('/stream');
    eventSource.onmessage = (event) => {
      const data = JSON.parse(event.data);
      updateUI(data);
    };

    function updateUI(data) {
      if (data.image) {
        img.src = `/static/${data.image}`;
        img.style.display = "block";
      }

      if (!data.animals_detected || data.animals_detected.length === 0) {
        resultsDiv.innerHTML = "✅<strong>No animals detected.</strong>";
      } else {
        resultsDiv.innerHTML = `<strong>Detected Animals:</strong><ul>` +
          data.animals_detected.map(a =>
            `<li>🦊<strong>${a.type}</strong> — ${a.confidence}%</li>`
          ).join("") +

```



```
`</ul>`;  
}  
}  
</script>  
</body>  
</html>
```

Appendix C: Bill of Material

Sr. No.	Name of components	Quantity	Price
1	Raspberry Pi 4 model B 1 GB RAM	1	5500
2	Raspberry Pi 5v power supply	1	600
3	HDMI Cable	1	180
4	SD Card & Formatter	1	500
5	USB Web Camera	1	550
6	Bread Board	1	50
7	PIR Sensor	1	50
8	Buzzer	1	25
9	LED	2	10
10	Jumping Wires	10	20
		Total	7480

